



Kubo

ERP + CRM autoalojable para PYMES
Documentación técnica y funcional

Versión 0.1.0-mvp · Armenia, Quindío · Licencia MIT
Generado el 2026-09-24

Documentación de Kubo

Documentación técnica y funcional del sistema. Está pensada para leerse en orden la primera vez y para consultarse por secciones después.

Contenido

| Documento | Para quién | Qué responde | | -- | -- | -- | | [01 – Arquitectura](01-arquitectura.md) | Técnico | ¿Cómo está construido y por qué así? Diagramas C4, flujos y atributos de calidad | | [02 – Modelo de datos](02-modelo-datos.md) | Técnico | ¿Qué se guarda, dónde y con qué reglas de integridad? | | [03 – API](03-api.md) | Desarrollador | ¿Qué endpoints existen y cómo se usan? Ejemplos con `curl` | | [04 – Seguridad](04-seguridad.md) | Técnico · Auditoría | ¿Cómo se protege la información y qué falta? | | [05 – Despliegue](05-despliegue.md) | Operación | ¿Cómo se instala, opera y respalda? | | [06 – Manual de usuario](06-manual-usuario.md) | Dueño del negocio | ¿Cómo se usa el sistema día a día? | | [07 – Pruebas](07-pruebas.md) | Calidad | ¿Qué se probó, cómo y con qué resultado? | | [08 – Trazabilidad](08-trazabilidad.md) | Evaluación | ¿Qué requisito se cumplió y dónde se comprueba? | | [09 – Guion de demostración](09-demo-guion.md) | Presentación | Guion del video de 6 minutos | | [10 – Auditoría técnica](10-auditoria.md) | Arquitectura · Calidad | Hallazgos, mediciones, correcciones aplicadas y catálogo de pendientes | | [11 – Plan de cierre](11-plan-de-cierre.md) | Gestión · Arquitectura | ¿Qué falta, en qué orden y con qué criterio se cierra cada fase? | | [ADRs](adr/) | Arquitectura | Las once decisiones que definen el sistema |

Decisiones de arquitectura (ADR)

| ADR | Decisión | | -- | -- | | [0001](adr/ADR-0001-microservicios-monolito-modular.md) | Microservicios con monolito modular interno | | [0002](adr/ADR-0002-polyrepo-contratos.md) | Polyrepo con contratos como fuente de verdad | | [0003](adr/ADR-0003-multitenancy-hibrido.md) | Multi-tenancy híbrido: `tenant\_id` + RLS activo | | [0004](adr/ADR-0004-poliglotismo.md) | Poliglotismo deliberado: un lenguaje por contexto | | [0005](adr/ADR-0005-eventos-rabbitmq.md) | Eventos con RabbitMQ y publicador tolerante a fallos | | [0006](adr/ADR-0006-cifrado-campos.md) | Cifrado de campos personales con AES-256-GCM | | [0007](adr/ADR-0007-jwt-rs256-jwks.md) | JWT RS256 con JWKS: la llave privada no sale del IAM | | [0008](adr/ADR-0008-alcance-mvp.md) | Alcance declarado del MVP y recortes conscientes | | [0009](adr/ADR-0009-outbox-transaccional.md) | Outbox transaccional para la entrega de eventos | | [0010](adr/ADR-0010-rls-activo.md) | RLS activo con interceptor de transacción | | [0011](adr/ADR-0011-bff-capa-formal.md) | BFF como capa formal del gateway | | [0012](adr/ADR-0012-zona-horaria-por-negocio.md) | Zona horaria por negocio | | [0013](adr/ADR-0013-vertical-packs.md) | Vertical Packs: el vertical como dato del negocio | | [0014](adr/ADR-0014-puerto-facturacion-dian.md) | Facturación electrónica DIAN como puerto enchufable | | [0015](adr/ADR-0015-segundo-factor-totp.md) | Segundo factor TOTP del propietario | | [0016](adr/ADR-0016-stock-por-bodega.md) | Stock por bodega y transferencias | | [0017](adr/ADR-0017-

puerto-notificaciones.md) | Puerto de notificaciones y buzón del negocio | | [0018](adr/ADR-0018-documentos.md) | Documentos en el ERP con puerto de almacenamiento |

Las siguientes ADR se registran al abrir cada paso de la Fase 4. Ver

11-plan-de-cierre.md.

Documentación por servicio

Cada repositorio tiene su propio README con lo específico:

- [kubo-gateway](../kubo-gateway/README.md) — enrutamiento, JWT, límites de tasa
- [kubo-iam](../kubo-iam/README.md) — identidad, tokens, auditoría
- [kubo-crm](../kubo-crm/README.md) — clientes y cifrado de campos
- [kubo-erp](../kubo-erp/README.md) — catálogo, kardex, ventas y eventos
- [kubo-analytics](../kubo-analytics/README.md) — proyección e indicadores
- [kubo-web](../kubo-web/README.md) — PWA y experiencia de usuario
- [kubo-infra](../kubo-infra/README.md) — orquestación, semilla y pruebas

Generar el PDF consolidado

```
./kubo-docs/scripts/build-pdf.sh
```

Genera `Kubo-Documentacion.pdf` en la raíz del workspace, con todos los documentos y los ADR unidos. Requiere Google Chrome o Chromium instalado.

01 — Arquitectura

Kubo es un ERP + CRM autoalojable para PYMES. Se instala en el local del negocio con un solo comando y opera como aplicación web instalable (PWA), incluso sin conexión a internet.

1. Vista de contexto (C4 nivel 1)

```
flowchart LR
```

```
DUENO["Dueño / Administrador"]
```

```
VENDEDOR["Vendedor en mostrador"]
```

```
KUBO["Kubo<br/>ERP + CRM autoalojable"]
```

```
CLIENTE["Cliente del negocio<br/>(WhatsApp / correo)"]
```

```
DUENO -->|"tablero, catálogo,<br/>clientes"| KUBO
```

```
VENDEDOR -->|"punto de venta"| KUBO
```

```
KUBO -. ->|"comprobantes y avisos<br/>(Fase 4)"| CLIENTE
```

2. Vista de contenedores (C4 nivel 2)

```

flowchart TB
    subgraph NAVEGADOR["Navegador del negocio"]
        PWA["PWA instalada<br/>React 19 + Vite<br/>cola offline en IndexedDB"]
    end

    subgraph SERVIDOR["Servidor del negocio (Docker Compose)"]
        TLS["kubo-tls<br/>Caddy · HTTPS"]
        WEB["kubo-web<br/>nginx · activos + proxy /api"]
        GW["kubo-gateway<br/>NestJS · API Gateway<br/>JWT RS256 · límites · BFF"]
        IAM["kubo-iam<br/>Java 21 · Spring Boot 4"]
        CRM["kubo-crm<br/>Ruby 3.4 · Rails 8"]
        ERP["kubo-erp<br/>Elixir 1.17 · Phoenix 1.8"]
        ANA["kubo-analytics<br/>Python 3.13 · FastAPI"]

        PG["PostgreSQL 17<br/>kubo_iam · kubo_crm · kubo_erp"]
        MG["MongoDB 8<br/>kubo_analytics"]
        MQ["RabbitMQ 4<br/>kubo.events"]
        RD["Redis 7<br/>límites de tasa"]
        OTEL["kubo-otel<br/>collector OTLP"]
    end

    PWA -->|"HTTPS 3443"| TLS
    TLS --> WEB
    WEB -->|"/api/v1"| GW
    GW -->|"verifica JWKS"| IAM
    GW --> IAM
    GW --> CRM
    GW --> ERP
    GW --> ANA
    GW --> RD

    IAM --> PG
    CRM --> PG
    ERP --> PG
    ANA --> MG

    ERP -->|"outbox: sale.created"| MQ
    MQ --> ANA

```

Responsabilidad de cada contenedor

| Contenedor | Responsabilidad | Base de datos | | -- | -- | -- | | `kubo-gateway` | Único punto de entrada: valida el JWT contra el JWKS, aplica límites de tasa, propaga la identidad y la trazabilidad, enruta | Redis (límites) | | `kubo-iam` | Negocios, usuarios, roles, tokens, auditoría | `kubo\_iam` | | `kubo-crm` | Clientes, cifrado de datos personales, cartera | `kubo\_crm` | | `kubo-erp` | Catálogo, inventario (kardex), ventas, anulación y **outbox** de eventos | `kubo\_erp` | | `kubo-analytics` | Modelo de lectura de ventas, indicadores del tablero | `kubo\_analytics` (MongoDB) | | `kubo-web` | Interfaz PWA; sirve los activos y proxea `/api` al gateway | -- | | `kubo-tls` | Terminación TLS (Caddy):

HTTPS, redirección de HTTP y HSTS | — || `kubo-otel` | Collector OpenTelemetry: recibe las trazas OTLP de los cinco servicios (opcional: Tempo + Grafana) | — |

3. Vista de componentes (C4 nivel 3, ejemplo: `kubo-erp`)

```
flowchart TB
    subgraph WEB["kubo-erp · capa de presentación"]
        ROUTER["Router /api/v1"]
        PLUG["Plug Identity<br/>(identidad del gateway)"]
        CTRL["ProductController<br/>SaleController<br/>StockController"]
        JSON["ProductJSON · SaleJSON"]
    end

    subgraph APP["Capa de aplicación"]
        CATALOG["Catalog<br/>catálogo e inventario"]
        SALES["Sales<br/>venta transaccional"]
        PUB["Events.Publisher<br/>(RabbitMQ)"]
        EVT["Events.SaleCreated"]
    end

    subgraph DOM["Capa de dominio"]
        PROD["Product"]
        MOVE["StockMovement"]
        SALE["Sale · SaleItem"]
    end

    subgraph INFRA["Infraestructura"]
        REPO["Ecto.Repo<br/>PostgreSQL"]
        BUS["AMQP"]
    end

    ROUTER --> PLUG --> CTRL --> JSON
    CTRL --> CATALOG
    CTRL --> SALES
    SALES --> PROD
    SALES --> SALE
    CATALOG --> PROD
    CATALOG --> MOVE
    SALES --> PUB
    PUB --> EVT
    PUB --> BUS
    CATALOG --> REPO
    SALES --> REPO
```

Los otros servicios siguen la misma separación. La regla es que el dominio no conoce HTTP, ni la base de datos, ni el bus de eventos.

4. Flujo crítico: registrar una venta

```
sequenceDiagram
    autonumber
    participant V as Vendedor (PWA)
    participant G as API Gateway
    participant E as kubo-erp
    participant P as PostgreSQL
    participant M as RabbitMQ
    participant A as kubo-analytics
    participant N as MongoDB

    V->>G: POST /api/v1/sales (Bearer JWT)
    G->>G: verifica firma RS256 contra el JWKS
    G->>E: POST /sales + X-User-Id, X-Tenant-Id
    E->>P: BEGIN
    E->>P: SELECT ... FOR UPDATE (productos)
    E->>P: INSERT sale, sale_items
    E->>P: UPDATE products.stock
    E->>P: INSERT stock_movements (kardex)
    E->>P: INSERT outbox_events (sale.created)
    E->>P: COMMIT
    E-->>G: 201 Created (venta)
    G-->>V: 201 Created
    Note over E,P: La venta y su evento se confirman juntos (outbox)
    E-)M: el publicador de barrido entrega sale.created
    M-)A: entrega del evento
    A->>N: deduplica por event_id y proyecta la venta
    Note over V,A: El tablero refleja la venta en menos de un segundo,<br/>sin que la caja
```

> Si el bus está caído, el evento ****espera en la bandeja de salida****

> (`outbox\_events`) y se entrega al volver; la venta ya está confirmada y el

> negocio sigue vendiendo. `make bus-drill` lo verifica (ver ADR-0009).

Si no hay internet en el local

```
sequenceDiagram
    autonumber
    participant V as Vendedor (PWA)
    participant Q as Cola IndexedDB
    participant G as API Gateway
    participant E as kubo-erp

    V->>Q: cobra sin conexión (venta guardada)
    Note over V,Q: La venta queda "pendiente de sincronizar"
    V->>V: vuelve la conexión (evento online)
    Q->>G: reintenta POST /sales con el token vigente
    G->>E: registra la venta
    E-->>Q: 201 Created
    Q->>Q: elimina la venta de la cola
```

5. Atributos de calidad y cómo se sostienen

| Atributo | Mecanismo | | --- | --- | | ****Escalabilidad**** | Servicios sin estado (excepto las bases); el gateway escala horizontalmente porque los límites de tasa viven en Redis. Cada base puede migrar a su propia instancia sin tocar código. | | ****Disponibilidad**** | Tolerancia a fallos en cadena: la venta no depende del bus ni de analítica; el gateway opera sin Redis (fail-open); cada contenedor tiene ***healthcheck*** y política de reinicio. | | ****Seguridad**** | JWT asimétrico, cifrado de campos, aislamiento por negocio, auditoría encadenada, límites de tasa y validación en cada frontera. | | ****Mantenibilidad**** | Un lenguaje por contexto con su ecosistema natural; arquitectura limpia dentro de cada servicio; contratos explícitos entre repos. | | ****Portabilidad**** | Todo corre en Docker Compose: el mismo artefacto sirve para el mini-PC del local y para un servidor en la nube. | | ****Observabilidad**** | Sonda `/health` en cada servicio con estado de sus dependencias, `correlation-id` propagado por el gateway y logs JSON estructurados. | | ****Costo**** | Consumo en reposo cercano a 1 GB de RAM y 12 contenedores; cabe en un VPS de USD 6–12 al mes o en un equipo modesto del local. El stack visual de trazas (Tempo + Grafana) es opcional. |

6. Decisiones arquitectónicas

Cada decisión relevante está registrada como ADR:

| ADR | Decisión | | --- | --- | | [0001](adr/ADR-0001-microservicios-monolito-modular.md) | Microservicios con monolito modular interno | | [0002](adr/ADR-0002-polyrepo-contratos.md) | Polyrepo con contratos como fuente de verdad | | [0003](adr/ADR-0003-multitenancy-hibrido.md) | Multi-tenancy híbrido (`tenant_id` + RLS activo) | | [0004](adr/ADR-0004-poliglotismo.md) | Poliglotismo deliberado: un lenguaje por contexto | | [0005](adr/ADR-0005-eventos-rabbitmq.md) | Eventos con RabbitMQ y publicador tolerante a fallos | | [0006](adr/ADR-0006-cifrado-campos.md) | Cifrado de campos personales con AES-256-GCM | | [0007](adr/ADR-0007-jwt-rs256-jwks.md) | JWT RS256 con JWKS en el gateway | | [0008](adr/ADR-0008-alcance-mvp.md) | Alcance declarado del MVP y recortes conscientes | | [0009](adr/ADR-0009-outbox-transaccional.md) | Outbox

transaccional para la entrega de eventos || [0010](adr/ADR-0010-rls-activo.md) | RLS activo con interceptor de transacción || [0011](adr/ADR-0011-bff-capa-formal.md) | BFF como capa formal del gateway || [0012](adr/ADR-0012-zona-horaria-por-negocio.md) | Zona horaria por negocio || [0013](adr/ADR-0013-vertical-packs.md) | Vertical Packs: el vertical como dato del negocio || [0014](adr/ADR-0014-puerto-facturacion-dian.md) | Facturación electrónica DIAN como puerto enchufable | || [0015](adr/ADR-0015-segundo-factor-totp.md) | Segundo factor TOTP del propietario || [0016](adr/ADR-0016-stock-por-bodega.md) | Stock por bodega y transferencias || [0017](adr/ADR-0017-puerto-notificaciones.md) | Puerto de notificaciones y buzón del negocio || [0018](adr/ADR-0018-documentos.md) | Documentos en el ERP con puerto de almacenamiento |

7. Estructura del workspace

```
Kubo/
├─ README.md           índice y arranque rápido
├─ Makefile            up · down · seed · smoke · pdf
├─ kubo-gateway/       API Gateway (TypeScript + NestJS)
├─ kubo-iam/           identidad (Java + Spring Boot)
├─ kubo-crm/           clientes (Ruby + Rails)
├─ kubo-erp/           catálogo, inventario y ventas (Elixir + Phoenix)
├─ kubo-analytics/     tablero (Python + FastAPI + MongoDB)
├─ kubo-web/           PWA (React + Vite + Tailwind)
├─ kubo-infra/         Docker Compose, semilla y prueba de humo
└─ kubo-docs/         esta documentación
```

02 — Modelo de datos

Kubo aplica **una base de datos por servicio**: ningún servicio consulta el

esquema de otro. Las referencias entre dominios son identificadores lógicos, no

llaves foráneas.

1. Panorama

```

flowchart LR
    subgraph PG["PostgreSQL 17 · una instancia, tres bases aisladas"]
        subgraph IAMDB["kubo_iam"]
            TENANTS["tenants"]
            USERS["users"]
            REFRESH["refresh_tokens"]
            RESET["password_reset_tokens"]
            MAIL["mail_outbox"]
            AUDIT["audit_logs"]
        end
        subgraph CRMDB["kubo_crm"]
            CUSTOMERS["customers"]
        end
        subgraph ERPDB["kubo_erp"]
            PRODUCTS["products"]
            MOVEMENTS["stock_movements"]
            SALES["sales"]
            ITEMS["sale_items"]
            OUTBOX["outbox_events"]
            COUNTERS["tenant_counters"]
        end
    end

    subgraph MONGO["MongoDB 8"]
        EVENTS["events<br/>(bitácora)"]
        PROJ["sales<br/>(proyección)"]
    end

    USERS -->|"tenant_id"| TENANTS
    REFRESH -->|"user_id"| USERS
    RESET -->|"user_id"| USERS
    CUSTOMERS -->|"tenant_id (lógico)"| TENANTS
    PRODUCTS -->|"tenant_id (lógico)"| TENANTS
    SALES -->|"sale_id"| ITEMS
    ITEMS -->|"product_id"| PRODUCTS
    MOVEMENTS -->|"product_id"| PRODUCTS
    SALES -->|"customer_id (lógico, sin FK)"| CUSTOMERS
    SALES -->|"sale.created (misma tx)"| OUTBOX
    OUTBOX -->|"publicador de barrido"| EVENTS
    EVENTS -->|"proyección"| PROJ

```

2. `kubo\_iam` — identidad y auditoría

```

erDiagram
    TENANTS ||--o{ USERS : "tiene"
    USERS ||--o{ REFRESH_TOKENS : "emite"
    USERS ||--o{ PASSWORD_RESET_TOKENS : "recupera"

    TENANTS {
        uuid id PK
        varchar name
        varchar slug UK
        varchar plan
        timestampz created_at
    }
    USERS {
        uuid id PK
        uuid tenant_id FK
        varchar email "único por negocio"
        varchar password_hash "BCrypt"
        varchar full_name
        varchar role "OWNER|ADMIN|SELLER|ACCOUNTANT|VIEWER"
        varchar status "ACTIVE|DISABLED"
        timestampz last_login_at
        integer failed_login_attempts "bloqueo por intentos"
        timestampz locked_until "temporal"
    }
    REFRESH_TOKENS {
        uuid id PK
        uuid user_id FK
        varchar token_hash UK "SHA-256, nunca el valor"
        timestampz expires_at
        timestampz revoked_at
        uuid replaced_by "rotación"
    }
    PASSWORD_RESET_TOKENS {
        uuid id PK
        uuid user_id FK
        varchar token_hash UK "SHA-256, un solo uso"
        timestampz expires_at "30 minutos"
        timestampz used_at
    }
    MAIL_OUTBOX {
        uuid id PK
        varchar recipient
        varchar subject
        text body "solo transporte log"
        varchar transport "log|smtp"
        timestampz created_at
    }
    AUDIT_LOGS {
        uuid id PK
        uuid tenant_id

```

```
uuid user_id
varchar action
varchar entity
varchar entity_id
varchar prev_hash
varchar hash "cadena SHA-256"
smallint hash_version "2 = vigente"
timestampz created_at
}
```

****Decisiones del modelo****

- `refresh\_tokens.token\_hash`: si la base se filtra, los tokens no se pueden reutilizar (solo se guarda su SHA-256).
- `replaced\_by`: al rotar un token se enlaza con el que lo reemplaza. Si vuelve a usarse uno revocado, se invalida toda la familia (detección de robo).
- `audit\_logs.prev\_hash` + `hash`: cada registro encadena el anterior; alterar un registro intermedio rompe la cadena y es detectable.
- `users.email` es único ****por negocio****, no global.
- `password\_reset\_tokens`: solo el SHA-256 del enlace, con expiración y uso único; al consumirse se revocan todas las sesiones del usuario.
- `users.failed\_login\_attempts` + `locked\_until`: el bloqueo por intentos vive en la misma fila y se levanta con un acceso correcto o una recuperación.
- `mail\_outbox`: buzón de demostración (transporte `log`); con `smtp` el cuerpo no se persiste.
- `audit\_logs.hash\_version`: cada fila declara con qué algoritmo se calculó su hash; las de la versión 1 solo se verifican por enlace.

3. `kubo\_crm` — clientes con datos personales cifrados

```

customers
├─ id                uuid  PK
├─ tenant_id         uuid  (aislamiento por negocio)
├─ name              varchar(160)
├─ email             varchar(180)
├─ document_number_encrypted text    ← AES-256-GCM: base64(iv || tag || ciphertext)
├─ document_number_bidx varchar(64) ← HMAC-SHA256 normalizado (búsqueda exacta)
├─ phone_encrypted   text    ← AES-256-GCM
├─ phone_bidx        varchar(64) ← HMAC-SHA256
├─ city, address, notes
├─ stage             LEAD | PROSPECT | CUSTOMER
├─ credit_limit       numeric(14,2) ≥ 0
├─ deleted_at         timestampz   (borrado lógico)
└─ created_at / updated_at

```

****Índices y restricciones****

| Objeto | Propósito | | -- | -- | | `idx\_customers\_tenant\_document\_unique` (único, parcial) | Un documento no puede repetirse dentro del mismo negocio; ignora borrados | |
 `idx\_customers\_tenant\_stage` | Filtros del tablero por etapa | | `customers\_stage\_check` | Solo los tres estados previstos | | `customers\_credit\_limit\_check` | El cupo no puede ser negativo |

4. `kubo\_erp` — catálogo, inventario y ventas

erDiagram

PRODUCTS ||--o{ STOCK_MOVEMENTS : "registra"

PRODUCTS ||--o{ SALE_ITEMS : "se vende en"

SALES ||--|{ SALE_ITEMS : "contiene"

PRODUCTS {

uuid id PK

uuid tenant_id

varchar sku "único por negocio"

varchar name

numeric price "precio final con IVA"

numeric cost

numeric tax_rate "19.00 por defecto"

integer stock "proyección del kardex"

integer min_stock "alerta de reposición"

boolean active

timestampz deleted_at

}

STOCK_MOVEMENTS {

uuid id PK

uuid tenant_id

uuid product_id FK

varchar kind "IN|OUT|ADJUST"

integer quantity "> 0"

integer stock_after

varchar reason

varchar reference_type "SALE|VOID|MANUAL"

uuid reference_id

uuid created_by

timestampz inserted_at

}

SALES {

uuid id PK

uuid tenant_id

varchar number "V-000001, único por negocio"

uuid customer_id "referencia lógica al CRM"

varchar customer_name "copia histórica"

varchar status "COMPLETED|VOIDED"

varchar payment_method "CASH|CARD|TRANSFER|CREDIT"

numeric subtotal

numeric tax

numeric total

uuid sold_by

timestampz voided_at

}

SALE_ITEMS {

uuid id PK

uuid sale_id FK

uuid tenant_id "denormalizado para RLS"

uuid product_id FK

```

    varchar product_name "copia histórica"
    integer quantity "> 0"
    numeric unit_price "copia histórica"
    numeric tax_rate
    numeric tax_amount
    numeric total
}
SUPPLIERS {
    uuid id PK
    uuid tenant_id
    varchar name
    varchar tax_id "NIT único por negocio"
    varchar contact_name
    varchar phone
    varchar email
    boolean active
    timestampz deleted_at
}
PURCHASES {
    uuid id PK
    uuid tenant_id
    varchar number "C-000001, único por negocio"
    uuid supplier_id FK
    varchar supplier_name "copia histórica"
    varchar status "RECEIVED|VOIDED"
    numeric subtotal
    numeric tax
    numeric total
    uuid received_by
    timestampz received_at
    timestampz voided_at
}
PURCHASE_ITEMS {
    uuid id PK
    uuid purchase_id FK
    uuid tenant_id "denormalizado para RLS"
    uuid product_id FK
    varchar product_name "copia histórica"
    integer quantity "> 0"
    numeric unit_cost "con IVA incluido"
    numeric tax_rate
    numeric tax_amount
    numeric total
}

```

****Reglas de integridad en el motor****

| Restricción | Qué garantiza | | -- | -- | | `products\_stock\_not\_negative` | El motor rechaza stock negativo aunque el código falle | | `stock\_movements\_reference\_unique` (único parcial) | Una venta no puede descontar dos veces el mismo producto (idempotencia) | | `sale\_items\_quantity\_positive` |

No existen líneas con cantidad cero o negativa | | `sales\_total\_not\_negative` | No existen ventas con total negativo | | `products\_tenant\_sku\_unique` (único parcial) | Un SKU no se repite dentro del negocio |

****Por qué el stock es una proyección****: la verdad está en `stock\_movements`.

`products.stock` se mantiene por velocidad de lectura y se actualiza ****dentro de la misma transacción**** que inserta el movimiento, con el producto bloqueado (`SELECT ... FOR UPDATE`). Ante cualquier duda se puede reconstruir el saldo sumando el kardex y compararlo con la proyección.

****Bandeja de salida (`outbox\_events`)****: el evento `sale.created` se guarda en la misma transacción de la venta (`status PENDING`), con `event\_id` único, el `payload` completo y `attempts`/`available\_at` para reintentos. El publicador de barrido lo marca `PUBLISHED` (o `FAILED` tras 10 intentos). Queda fuera de RLS a propósito: se lee cruzando negocios para entregar los eventos (ADR-0009).

****Numeración atómica (`tenant\_counters`)****: los consecutivos de venta (`sale\_seq`) y de compra (`purchase\_seq`) se obtienen con un UPSERT (`ON CONFLICT DO UPDATE ... RETURNING`), sin leer el máximo ni reintentar; dos cajas nunca reciben el mismo número. La tabla no lleva RLS: es numeración operativa, sin datos de negocio. `sale\_items.tenant\_id` y `purchase\_items.tenant\_id` se denormalizan desde su documento para que las políticas de RLS sean comparaciones por índice (la subconsulta anterior fallaba de forma intermitente bajo concurrencia).

****Compras y proveedores (Fase 3)****: `suppliers` guarda el proveedor con su NIT (único por negocio y borrado lógico); `purchases` la cabecera (número `C-000001`, proveedor con copia del nombre, totales y estado `RECEIVED|VOIDED`) y `purchase\_items` el detalle con el costo unitario. La compra suma inventario, actualiza el costo del producto sin IVA y emite `purchase.received`.

5. `kubo\_analytics` – modelo de lectura en MongoDB

```
events (bitácora cruda)
├─ _id          = event_id del evento ← índice único: deduplicación
├─ event_type   "sale.created"
├─ version      1
├─ tenant_id
├─ occurred_at  date
├─ payload      documento original completo
└─ received_at  date

sales (proyección consultable)
├─ _id          ObjectId
├─ sale_id      índice único
├─ tenant_id    + sold_at (índice compuesto)
├─ number, status, payment_method
├─ customer_id, customer_name
├─ subtotal, tax, total      (double: analítica, no contabilidad)
├─ items[]         {product_id, product_name, quantity, unit_price, tax_amount, total}
└─ sold_at, created_at, updated_at
```

****Idempotencia****: `events.\_id` es el `event\_id`. Si el mismo evento llega dos

veces, la segunda inserción falla con clave duplicada y se descarta: una venta

nunca se cuenta dos veces.

****Precisión numérica****: el sistema de registro contable es `kubo-erp`, con

aritmética decimal. Aquí los importes se guardan como punto flotante porque las

agregaciones sobre miles de documentos no necesitan precisión de centavo y así

se aprovechan los operadores nativos de MongoDB. Es una decisión consciente: el

tablero es para decidir, la factura es para cobrar.

6. Convenciones transversales

| Convención | Decisión | | --- | --- | | Identificadores | UUID (v4) en todas las tablas; nunca enteros autoincrementales expuestos | | Fechas | `timestampz` en PostgreSQL, almacenadas en UTC; la interfaz muestra `America/Bogota` | | Dinero | `numeric(14,2)` y aritmética decimal en el ERP; nunca coma flotante | | Borrado | Lógico (`deleted\_at`), nunca físico: la historia comercial se conserva | | Aislamiento | `tenant\_id` en toda tabla de negocio + ****RLS activo con `FORCE`**** en las tres bases (ADR-0010) | | Migraciones | Versionadas y ****inmutables**** una vez aplicadas; nunca se edita una migración ya ejecutada |

03 — API

Todas las peticiones entran por el **API Gateway** (`http://localhost:9080` en desarrollo). El gateway valida el JWT, aplica límites de tasa y enruta al servicio correspondiente. Ningún servicio de negocio está expuesto directamente.

- **Base**: `/api/v1`
- **Formato**: JSON UTF-8
- **Autenticación**: `Authorization: Bearer <access token>`
- **Trazabilidad**: cabecera `X-Correlation-Id` (si no llega, el gateway la crea)

1. Rutas públicas y protegidas

Ruta	Acceso	Metodo	Protegido
<code>/api/v1/health</code>	Público	GET	No
<code>/api/v1/auth/login</code>	Público	POST	No
<code>/api/v1/auth/register</code>	Público	POST	No
<code>/api/v1/auth/refresh</code>	Público (cookie de refresco)	POST	No
<code>/api/v1/auth/logout</code>	Público (cookie de refresco)	POST	No
<code>/api/v1/auth/forgot-password</code>	Público	POST	No
<code>/api/v1/auth/reset-password</code>	Público	POST	No
<code>/api/v1/auth/.well-known/jwks.json</code>	Público	GET	No
Todo lo demás	Requiere <code>Bearer</code> válido		Si

2. Enrutamiento

Prefijo	Servicio
<code>/api/v1/auth</code> , <code>/api/v1/users</code> , <code>/api/v1/audit</code>	kubo-iam
<code>/api/v1/customers</code>	kubo-crm
<code>/api/v1/products</code> , <code>/api/v1/sales</code> , <code>/api/v1/stock</code>	kubo-erp
<code>/api/v1/dashboard</code> , <code>/api/v1/events</code>	kubo-analytics

3. Identidad (`kubo-iam`)

`POST /auth/register`

Crea un negocio y su usuario propietario.

```
curl -X POST http://localhost:9080/api/v1/auth/register \
-H 'Content-Type: application/json' \
-d '{
  "tenantName": "Tienda La Esquina",
  "fullName": "Administrador",
  "email": "admin@kubo.local",
  "password": "Admin123!"
}'
```

`POST /auth/login`

```
curl -X POST http://localhost:9080/api/v1/auth/login \
-H 'Content-Type: application/json' \
-d '{"email":"admin@kubo.local","password":"Admin123!"}'
```

Respuesta:

```
{
  "accessToken": "eyJhbGciOiJSUzI1NiIsImtpZCI6...\"",
  "tokenType": "Bearer",
  "expiresInSeconds": 900,
  "user": {
    "id": "...", "email": "admin@kubo.local", "fullName": "Administrador Kubo",
    "role": "OWNER", "tenantId": "...", "tenantName": "Tienda La Esquina"
  }
}
```

El **access token** dura 15 minutos y se firma con RS256. El **refresh token** dura 7 días, se guarda hasheado y **rota en cada uso**; reutilizar uno revocado invalida toda la familia de tokens del usuario. El gateway (BFF) lo entrega en una **cookie `httpOnly` + `SameSite=Strict`** (`Path=/api/v1/auth`) y **nunca** aparece en el cuerpo de la respuesta: un XSS no puede robarlo.

`POST /auth/forgot-password`

Solicita el enlace de recuperación. Responde siempre `204`, exista o no la cuenta (no permite enumerar usuarios).

```
curl -X POST http://localhost:9080/api/v1/auth/forgot-password \
-H 'Content-Type: application/json' \
-d '{"email":"admin@kubo.local"}'
```

Con `KUBO\_MAIL\_TRANSPORT=log` (demostración) el correo queda en la tabla `mail\_outbox`; con `smtp` sale por el servidor configurado.

`POST /auth/reset-password`

Consume el token del enlace (un solo uso, vence en 30 minutos) y cambia la contraseña; además revoca todas las sesiones y levanta el bloqueo por intentos.

```
curl -X POST http://localhost:9080/api/v1/auth/reset-password \
-H 'Content-Type: application/json' \
-d '{"token":"<token del enlace>","newPassword":"NuevaClave123!"}'
```

`GET /auth/me`

Devuelve el perfil del usuario autenticado.

`GET /auth/.well-known/jwks.json`

Llave pública RSA que el gateway usa para verificar firmas.

4. Clientes (`kubo-crm`)

| Método | Ruta | Descripción | | -- | -- | -- | | GET | `/customers?q=&stage=` | Listado (documento y teléfono ****enmascarados****) | | GET | `/customers/{id}` | Detalle (valores descifrados) | | GET | `/customers/by-document/{documento}` | Búsqueda exacta por documento (índice ciego) | | POST | `/customers` | Crear | | PATCH | `/customers/{id}` | Actualizar | | DELETE | `/customers/{id}` | Borrado lógico | | GET | `/customers/stats` | Conteos por etapa y cartera |

```
TOKEN=$(curl -s -X POST http://localhost:9080/api/v1/auth/login \
-H 'Content-Type: application/json' \
-d '{"email":"admin@kubo.local","password":"Admin123!"}' | jq -r .accessToken)

curl -X POST http://localhost:9080/api/v1/customers \
-H "Authorization: Bearer ${TOKEN}" -H 'Content-Type: application/json' \
-d '{"name":"Panadería El Trigal","document_number":"900123456-7",
  "phone":"3105551234","city":"Armenia","stage":"CUSTOMER}"'
```

5. Catálogo, inventario y ventas (`kubo-erp`)

| Método | Ruta | Descripción | | -- | -- | -- | | GET | `/products?q=&low\_stock=` | Catálogo | | POST | `/products` | Crear producto | | PATCH | `/products/{id}` | Actualizar | | DELETE | `/products/{id}` | Borrado lógico | | GET | `/products/stats` | Totales y valor del inventario | | POST | `/products/{id}/stock` | Movimiento de inventario | | GET | `/stock/movements?product\_id=` | Kardex | | GET | `/sales?status=` | Ventas | | POST | `/sales` | Registrar venta | | GET | `/sales/{id}` | Detalle | | POST | `/sales/{id}/void` | Anular venta (devuelve inventario) | | GET | `/sales/stats` | Totales del día | | GET | `/suppliers?q=&active=` | Proveedores (paginado) | | POST | `/suppliers` | Crear proveedor | | PATCH | `/suppliers/{id}` | Actualizar proveedor | | DELETE | `/suppliers/{id}` | Borrado lógico | | GET | `/purchases?status=&supplier\_id=` | Compras (paginadas) | | POST | `/purchases` | Registrar compra (suma inventario) | | GET | `/purchases/{id}` | Detalle de compra | | POST | `/purchases/{id}/void` | Anular compra (revierte inventario) | | GET | `/purchases/stats` | Totales de compras y proveedores |

Registrar una compra

```
curl -X POST http://localhost:9080/api/v1/purchases \
-H "Authorization: Bearer ${TOKEN}" -H 'Content-Type: application/json' \
-d '{
  "supplier_id": "...",
  "items": [{"product_id": "...", "quantity": 5, "unit_cost": 5000}]
}'
```

La compra **suma** el inventario, deja el movimiento en el kardex

(`reference\_type = PURCHASE`) y actualiza `products.cost` con el valor **sin**

IVA **(el costo unitario se interpreta con IVA incluido, igual que el precio de**

venta). `POST /purchases/{id}/void` la anula y revierte el stock.

Registrar una venta

```
curl -X POST http://localhost:9080/api/v1/sales \
-H "Authorization: Bearer ${TOKEN}" -H 'Content-Type: application/json' \
-d '{
  "items": [
    {"product_id": "...", "quantity": 2},
    {"product_id": "...", "quantity": 1}
  ],
  "customer_id": "...",
  "customer_name": "Panadería El Trigal",
  "payment_method": "CASH"
}'
```

Respuesta `201`:

```
{
  "data": {
    "id": "...", "number": "V-000004", "status": "COMPLETED",
    "subtotal": "31680.67", "tax": "6019.33", "total": "37700.00",
    "items": [{ "product_name": "Cafe molido 250 g", "quantity": 2,
      "unit_price": "9500.00", "tax_amount": "3033.61", "total": "19000.00" }]
  }
}
```

Reglas de negocio aplicadas

| Situación | Respuesta | | -- | -- | | Venta sin líneas | `422 EMPTY\_SALE` | | Producto inexistente o de otro negocio | `422 PRODUCT\_NOT\_FOUND` | | Cantidad mayor al stock | `409 INSUFFICIENT\_STOCK` (con el stock disponible) | | Venta ya anulada | `409 ALREADY\_VOIDED` | | Colisión del número de venta | reintento automático (hasta 3) |

Ajustar inventario

```
curl -X POST http://localhost:9080/api/v1/products/{id}/stock \
-H "Authorization: Bearer ${TOKEN}" -H 'Content-Type: application/json' \
-d '{"kind": "IN", "quantity": 40, "reason": "Compra a proveedor"}'
```

`kind` acepta `IN` (entrada), `OUT` (salida) y `ADJUST` (fijar el stock exacto).

6. Tablero (`kubo-analytics`)

| Método | Ruta | Descripción | | -- | -- | -- | | GET | `/dashboard/overview` | ****Vista compuesta (BFF)****: el gateway consulta en paralelo las 7 vistas y devuelve una sola respuesta | | GET | `/dashboard/summary` | Ventas, ingreso, IVA, ticket promedio, unidades, hoy | | GET | `/dashboard/sales-by-day?days=14` | Serie diaria (en la zona horaria del negocio) | | GET | `/dashboard/top-products?limit=10` | Productos más vendidos | | GET | `/dashboard/top-customers?limit=10` | Mejores clientes | | GET | `/dashboard/payment-methods` | Distribución por medio de pago | | GET | `/dashboard/recent-sales?limit=10` | Últimas ventas | | GET | `/dashboard/rotation` | Productos con más salida en 7 días | | POST | `/events` | Vía de respaldo para reenviar eventos por HTTP |

Vista compuesta (patrón BFF)

```
curl http://localhost:9080/api/v1/dashboard/overview -H "Authorization: Bearer ${TOKEN}"
```

```
{
  "data": {
    "summary": { "sales_count": 10, "revenue": 141900.0, "today": { "revenue": 0, "sales_count": 0 }, "sales_by_day": [ { "date": "2026-09-23", "revenue": 141900.0, "sales_count": 10 } ], "top_products": [ { "product_name": "Cafe molido 250 g", "quantity": 6, "revenue": 5700.0 }, { "product_name": "Panela 500 g", "quantity": 4, "revenue": 2000.0 } ], "payment_methods": [ { "payment_method": "CASH", "revenue": 45300.0, "sales_count": 3 }, { "payment_method": "CARD", "revenue": 96600.0, "sales_count": 7 } ], "recent_sales": [ { "number": "V-000010", "total": 35700.0 } ], "rotation": [ { "product_name": "Panela 500 g", "quantity_7d": 4 } ], "customers": { "total": 11, "by_stage": { "LEAD": 8, "PROSPECT": 1, "CUSTOMER": 2 } } }
  }
}
```

Por qué existe: sin esta capa la PWA necesitaba ****siete viajes de red**** para pintar el tablero (108 ms medidos). El gateway consulta los servicios en paralelo dentro de la red privada y responde en ~25 ms. Si una vista falla, la respuesta incluye `unavailable: ["top\_products"]` y el tablero se muestra parcial en lugar de caer.

Zona horaria del negocio

Los indicadores de «hoy» y la serie diaria se calculan en la zona horaria del negocio (`KUBO\_TIMEZONE`, por defecto `America/Bogota`), no en UTC. Sin esto, en Colombia las ventas de 19:00 a 23:59 se atribuirían al día siguiente. La respuesta de `/sales/stats` incluye `timezone` y `business\_date` para poder auditarlo.

7. Eventos de dominio

El ERP publica en el exchange `*topic*`kubo.events``. Sobre del evento:

```
{
  "event_id": "9f1c...",
  "event_type": "sale.created",
  "version": 1,
  "occurred_at": "2026-09-23T17:50:12.480Z",
  "tenant_id": "...",
  "data": {
    "sale_id": "...", "number": "V-000004", "status": "COMPLETED",
    "customer_id": "...", "customer_name": "Panadería El Trigal",
    "payment_method": "CASH",
    "subtotal": "31680.67", "tax": "6019.33", "total": "37700.00",
    "sold_by": "...", "sold_at": "2026-09-23T17:50:12.470Z",
    "items": [
      { "product_id": "...", "product_name": "Cafe molido 250 g", "quantity": 2,
        "unit_price": "9500.00", "tax_rate": "19.00",
        "tax_amount": "3033.61", "total": "19000.00" }
    ]
  }
}
```

****Contrato de consumo****: el consumidor debe deduplicar por `event\_id` y tolerar campos nuevos (versionado aditivo). Los mensajes fallidos van a la cola ``kubo.analytics.sale_created.dlq``.

8. Formato de errores

Todos los servicios responden con la misma forma:

```
{
  "code": "INSUFFICIENT_STOCK",
  "message": "Stock insuficiente de Cafe molido 250 g: disponible 7"
}
```

| Código HTTP | Cuándo | | -- | -- | | `400` | Datos inválidos (`VALIDATION\_ERROR`,
`INVALID\_QUANTITY`) | | `401` | Sin token, token inválido/expirado, identidad ausente | | `404` |
Recurso inexistente o de otro negocio | | `409` | Conflicto de estado (`INSUFFICIENT\_STOCK`,
`ALREADY\_VOIDED`, `EMAIL\_ALREADY\_EXISTS`) | | `422` | Regla de negocio incumplida
(`EMPTY\_SALE`, `PRODUCT\_NOT\_FOUND`) | | `429` | Límite de tasa excedido | | `502` | Microservicio
no disponible (`UPSTREAM\_UNAVAILABLE`) |

Los mensajes nunca incluyen trazas ni detalles internos; los detalles técnicos

quedan en los logs con el `correlation-id` de la petición.

9. Límites de tasa

| Ámbito | Límite por defecto | | -- | -- | | Rutas de negocio | 600 peticiones/minuto por negocio (o por
IP si no hay token) | | Rutas de autenticación | 40 peticiones/minuto (frena fuerza bruta) |

Las respuestas incluyen `X-RateLimit-Limit` y `X-RateLimit-Remaining`. Al

excederse: `429 RATE\_LIMIT\_EXCEEDED`.

04 — Seguridad

Kubo maneja datos personales de clientes, precios de compra y cartera: la

seguridad no es un añadido, es parte del diseño. Este documento explica **qué se**

implementó, cómo se verifica y qué queda pendiente**.**

1. Modelo de confianza

```
flowchart LR
    EXT["Red del local<br/>(posible red compartida)"]
    GW["API Gateway<br/>único punto expuesto"]
    NET["Red privada de contenedores<br/>kubo-net"]
    SVC["Microservicios<br/>sin puertos públicos"]

    EXT -->|"HTTPS + JWT"| GW
    GW -->|"cabeceras de identidad<br/>X-User-Id · X-Tenant-Id"| NET
    NET --> SVC
```

- Solo el gateway (9080) y la PWA (3000) se publican. Los servicios 9081–9084 solo se mapean al host para diagnóstico local; **en producción no se publican**.
- La identidad se establece **una sola vez**, en el gateway. Los servicios internos confían en las cabeceras porque no son alcanzables desde fuera.
- El gateway **elimina cualquier cabecera `X-User-`** que envíe el cliente **antes de inyectar la identidad verificada**: sin esto, cualquiera podría suplantar a otro usuario con un `curl`.

2. Autenticación

| Control | Implementación | | --- | --- | | Contraseñas | **BCrypt** (coste 10). Nunca se persisten ni se registran en logs | | Access token | **JWT RS256**, 15 minutos, con `iss`, `aud`, `jti`, `exp` y claims de negocio (`tenant\_id`, `role`) | | Verificación | Solo el gateway, contra el **JWKS** del servicio de identidad, con caché y refresco automático | | Refresh token | Valor aleatorio de 384 bits; se guarda **solo su SHA-256** | | Rotación | Cada refresh emite uno nuevo y revoca el anterior | | Detección de reuso | Si se usa un token ya rotado, se **invalida toda la familia** del usuario y se audita `REFRESH\_REUSE\_DETECTED` | | Cierre de sesión | Revoca el refresh token en el servidor, no solo en el navegador | | Almacenamiento del refresh | **Cookie `httpOnly` + `SameSite=Strict`** emitida por el gateway (BFF); el cuerpo de la respuesta nunca lo incluye | | Bloqueo de cuenta | Tras 5 intentos fallidos la cuenta se bloquea 15 minutos; el acceso correcto o la recuperación lo levantan | | Recuperación de contraseña | Enlace de un solo uso, hashado en base, vence en 30 minutos; al usarse revoca todas las sesiones |

Por qué asimétrico: con un secreto compartido, comprometer cualquier servicio permitiría *emitir* tokens válidos para cualquier usuario. Con RS256, la llave privada vive solo en `kubo-iam`.

3. Cifrado

| Ámbito | Mecanismo | Dónde se verifica | | --- | --- | --- | | En tránsito (externo) | TLS terminado en Caddy (`kubo-tls`, puertos 3080/3443) con HSTS y redirección de HTTP | | `make smoke` bloque 10 · `05-despliegue.md` §3 | | En tránsito (interno) | Red privada de contenedores; mTLS previsto para clúster | — | | **Datos personales** | **AES-256-GCM** campo a campo (documento, teléfono) | | `make smoke` inspecciona el texto cifrado en PostgreSQL | | Búsqueda sobre cifrado | **Índice ciego** HMAC-SHA256 normalizado | | `make smoke` verifica que el índice no contiene el valor | | Contraseñas | BCrypt con sal por usuario | — | | Respaldos | Cifrado con `age` antes de salir del servidor | Implementado: `make backup` / `make restore-drill` |

Formato almacenado: ``base64(iv || tag || ciphertext)``. El **tag** de GCM hace que cualquier manipulación del texto cifrado sea detectable: el descifrado falla y el servicio devuelve ``null`` en lugar de un valor corrupto.

Gestión de claves

- Las claves llegan por variable de entorno y **nunca** están en el repositorio (``.env`` está en ``.gitignore``; solo se versiona ``.env.example`` con ceros).
- ``KUBO_FIELD_ENCRYPTION_KEY`` y ``KUBO_BLIND_INDEX_KEY`` son de 32 bytes en hex; el servicio **falla al arrancar** si no cumplen el formato, en lugar de cifrar con una clave débil.
- El servicio **rechaza explícitamente** las claves de ejemplo de ``.env.example``: desplegar copiando la plantilla sin generar claves produce un error claro (``«conserva el valor de ejemplo; genere una clave real»``) en vez de proteger los datos con una clave conocida por cualquiera. Cubierto por dos pruebas unitarias.
- La llave de firma JWT puede llegar como PEM; si no, el servicio genera un par **efímero** en memoria y lo advierte en los logs (solo desarrollo).
- **Pendiente**: envoltura KEK/DEK por negocio con KMS para la versión en nube, y rotación de claves de campo (exige re-cifrado por lotes).

4. Aislamiento entre negocios

1. Toda tabla de negocio lleva ``tenant_id`` y toda consulta lo filtra.
2. El ``tenant_id`` **nunca** viene del cuerpo ni de un parámetro: se toma del claim del JWT, que el gateway propaga como cabecera.
3. **RLS activo con `FORCE` en las tres bases** (P-02, ADR-0010). Cada petición abre una transacción y fija ``app.tenant_id`` con ``set_config(..., true)``; sin contexto, una consulta devuelve **cero filas**. Las operaciones que cruzan

negocios por diseño (autenticación por correo, cadena de auditoría, semilla, respuesta a incidentes) usan la marca `app.system`, deliberada y acotada.

4. **Verificación automática**: `make smoke` comprueba que sin contexto no hay filas y con contexto sí, en IAM, CRM y ERP, además de registrar un segundo negocio y comprobar que no ve ni un cliente ni un producto del primero.

5. Autorización

Roles definidos: `OWNER`, `ADMIN`, `SELLER`, `ACCOUNTANT`, `VIEWER`. El rol viaja en el token y el gateway lo propaga; cada servicio decide qué permite. En el MVP el control es por rol y negocio (no hay permisos finos por recurso), lo que se documenta como alcance.

6. Auditoría

`kubo-iam` mantiene `audit\_logs` **append-only** con cadena de hash: cada registro incluye el hash del anterior, de modo que alterar o borrar un registro intermedio rompe la cadena y es detectable. Se auditan, entre otros: registro de negocio, inicio de sesión correcto y fallido, rotación de token, reuso detectado.

Serialización de la cadena: leer el último hash e insertar el nuevo debe ser atómico. Sin un bloqueo, dos peticiones concurrentes leerían el mismo hash anterior y la cadena quedaría partida en dos ramas, lo que haría indetectable una manipulación posterior. `AuditService` toma un `pg\_advisory\_xact\_lock` al inicio de la transacción: se libera solo al terminar y no bloquea ninguna tabla.

Los logs son **JSON estructurados** y el gateway **redacta** la cabecera `Authorization` y las cookies. Nunca se registran contraseñas, tokens ni datos personales: Rails filtra `document\_number`, `phone` y `email` en sus logs.

7. Protección de la interfaz y la API

| Amenaza | Mitigación | | --- | --- | | Fuerza bruta de contraseñas | Límite de 40 peticiones/minuto en rutas de autenticación + auditoría de intentos fallidos | | Abuso de la API | Límite de 600 peticiones/minuto por negocio o IP | | XSS | React escapa por defecto; no se usa `dangerouslySetInnerHTML`; cabeceras de seguridad con Helmet | | Clickjacking | `X-Frame-Options` de Helmet | | MIME sniffing | `X-Content-Type-Options: nosniff` | | Inyección SQL | Consultas parametrizadas en los cuatro lenguajes (JPA, ActiveRecord, Ecto, PyMongo); ****no hay SQL concatenado**** | | Inyección en búsquedas | Los comodines `%` y `\_` se eliminan del término de búsqueda | | CSRF | La API no usa cookies de sesión: la autenticación es por cabecera `Authorization` | | Enumeración de usuarios | El login responde el mismo mensaje para correo inexistente y contraseña incorrecta | | Fuga de datos entre negocios | Aislamiento por `tenant\_id` + prueba automática |

Sobre el almacenamiento del refresh token

El refresh token ****no toca el navegador****: el gateway (BFF) lo recibe del IAM y lo deja en una cookie `httpOnly` + `SameSite=Strict` con `Path=/api/v1/auth`, y lo elimina del cuerpo de la respuesta (P-03, verificado en el humo). El access token (15 minutos) vive solo en memoria. El refresh rota en cada uso y el reuso se detecta invalidando la familia completa: el humo comprueba que, tras un reuso, ni la cookie rotada sigue sirviendo. En producción con HTTPS, `KUBO\_COOKIE\_SECURE=true` marca además la cookie como `Secure`.

8. Cumplimiento normativo

| Norma | Cómo se aborda | | --- | --- | | ****Ley 1581 de 2012**** (protección de datos personales, Colombia) | Cifrado de datos personales, minimización (enmascarado en listados), borrado lógico reversible, trazabilidad de accesos. ****Pendiente****: registro de autorizaciones del titular, política de retención y procedimiento de supresión definitiva | | ****Facturación electrónica DIAN**** | Fuera del alcance del MVP (requiere ser Proveedor Tecnológico autorizado). El dominio deja preparado el puerto de facturación para emitir UBL 2.1 con CUFE y QR en la Fase 4 (P-18) | | ****Habeas data en la interfaz**** | El manual de usuario documenta qué datos se guardan, para qué y cómo se solicitan al cliente |

9. Verificación

```
make smoke
```

La prueba de humo comprueba, entre otras cosas:

- Petición sin token → `401`.
- Token con firma falsificada → `401`.
- Cabecera `X-User-Id` inyectada por el cliente → **ignorada** (`401`).
- Documento cifrado en PostgreSQL y legible solo por el detalle autorizado.
- Listado con el documento enmascarado.
- Índice ciego sin el valor en claro.
- Segundo negocio que no ve datos del primero.

10. Pendientes de seguridad

| Pendiente | ID · Fase | Riesgo que cierra | | --- | --- | --- | | Segundo factor (TOTP) para el propietario | P-30 · Fase 4 | Robo de credenciales | | mTLS entre gateway y servicios | P-28 · Fase 5 | Movimiento lateral dentro del clúster | | Rotación de claves de cifrado de campo | P-29 · Fase 5 | Compromiso de una clave a largo plazo |

> **Resueltos**: en la Fase 1, cookie `httpOnly` (P-03), RLS activo (P-02) y

> límite de tasa por usuario (P-31); en la Fase 2, SAST/escaneo de secretos y

> dependencias en CI (P-06) y contratos ejecutables (P-09). Ver

> [`11-plan-de-cierre.md`](11-plan-de-cierre.md).

05 — Despliegue y operación

1. Requisitos

| Recurso | Mínimo | Recomendado | | --- | --- | --- | | CPU | 2 núcleos | 4 núcleos | | RAM | 4 GB | 8 GB | | Disco | 20 GB | 60 GB SSD | | Sistema | Linux con Docker Engine 24+ y el plugin `docker compose` | Ubuntu 24.04 LTS | | Red | Salida a internet solo para construir/actualizar | — |

El sistema completo consume **menos de 2 GB de RAM en reposo** (límites por contenedor definidos en `kubo-infra/docker-compose.yml`).

2. Instalación en el local del negocio

```
git clone <url-del-workspace> kubo && cd kubo
cp kubo-infra/.env.example kubo-infra/.env
```

Editar `kubo-infra/.env` y **cambiar todos los valores**. Generar las claves:

```
openssl rand -hex 32 # KUBO_FIELD_ENCRYPTION_KEY
openssl rand -hex 32 # KUBO_BLIND_INDEX_KEY
openssl rand -hex 64 # KUBO_CRM_SECRET_KEY_BASE
openssl rand -base64 48 # KUBO_ERP_SECRET_KEY_BASE
```

Levantar:

```
make up # construye y arranca los 12 contenedores
make seed # datos de demostración (omitir en producción)
make smoke # verificación end-to-end
```

Abrir `http://<ip-del-servidor>:3000` e ingresar con el administrador creado.

Puertos

Servicio	Host	Contenedor	---	---	---	PWA	3000	80	**TLS (Caddy): HTTPS · HTTP**
			3443 · 3080	443 · 80	**OTel collector: gRPC · HTTP**	**4317 · 4318**	4317 · 4318		
Grafana (perfil `observability`)			3001	3000	API Gateway	9080	8080		IAM · CRM · ERP ·
Analítica	9081–9084	8081–8084			PostgreSQL · MongoDB	5433 · 27018	5432 · 27017		
RabbitMQ (AMQP · consola)	5673 · 15673	5672 · 15672			Redis	6380	6379		

Los puertos se eligieron **fuera del rango 8000–8100** para no chocar con otros servicios que ya corran en la máquina del negocio.

3. Puesta en producción (checklist)

- [] `kubo-infra/.env` con todas las claves cambiadas y el archivo **fuera del control de versiones**.
- [] `KUBO\_JWT\_PRIVATE\_KEY` con una llave RSA real (si queda vacía, el servicio genera una efímera y los tokens se invalidan al reiniciar).
- [] TLS: `KUBO\_TLS\_DOMAIN` con el dominio del negocio (ver abajo). El servicio `kubo-tls` ya está en el compose.
- [] `KUBO\_COOKIE\_SECURE=true` cuando el acceso sea siempre por HTTPS.
- [] Correo real: `KUBO\_MAIL\_TRANSPORT=smtp` con `KUBO\_SMTP\_HOST` y credenciales (si se deja `log`, el enlace de recuperación queda en el buzón de demostración).
- [] **No publicar** los puertos 9080–9084 en producción: la PWA llega al gateway por la red interna de contenedores (nginx proxea `/api`), y los servicios de negocio solo deben ser alcanzables por el gateway. Publicar el 9080 permitiría inyectar `X-Forwarded-For` y saltarse el límite de autenticación.

- [] `KUBO\_SEED\_ENABLED=false` (no sembrar datos de demostración).
- [] RespalDOS programados (sección 5) y un simulacro de restauración ejecutado.
- [] Monitoreo de las sondas `/api/v1/health` de los cinco servicios.

TLS con Caddy (incluido)

El servicio `kubo-tls` (Caddy) termina HTTPS en el local y redirige HTTP a HTTPS:

- `https://<dominio>:3443` sirve la PWA; `http://<dominio>:3080` redirige.
- Con `KUBO\_TLS\_DOMAIN=localhost` (por defecto) Caddy usa su CA interna (`tls internal`): válido para la red del local, el navegador pedirá aceptar el certificado una vez.
- Para un **dominio público**: ponga `KUBO\_TLS\_DOMAIN=kubo.minegocio.co` en el `.env`, quite `tls internal` de `kubo-infra/caddy/Caddyfile` y Caddy gestionará el certificado Let's Encrypt automáticamente.

La PWA ya proxea `/api` al gateway, así que un único dominio con HTTPS cubre todo el sistema. La cabecera `Strict-Transport-Security` se envía en todas las respuestas.

Trazas (OpenTelemetry)

El collector `kubo-otel` recibe trazas OTLP de los cinco servicios y las escribe en su log: `docker logs kubo-otel` es la comprobación básica (el humo la verifica).

Para el stack visual:

```
make observability # agrega Tempo y Grafana; Grafana en http://localhost:3001
```

No forma parte de la instalación por defecto: un local de barrio no necesita el stack visual y así el consumo se mantiene bajo. En producción, fije la versión de las imágenes de observabilidad (hoy usan etiquetas).

4. Operación diaria

| Comando | Para qué | | -- | -- | | `make ps` | Ver el estado y la salud de cada contenedor | | `make logs` | Logs en vivo de todos los servicios | | `make restart` | Reiniciar los servicios de aplicación | | `make smoke` | Verificar el flujo completo (útil tras una actualización) | | `make down` | Detener el sistema conservando los datos | | `make clean` | Detener ***y borrar*** los volúmenes (pierde los datos) |

Actualización

```
git pull
make up      # reconstruye solo lo que cambió
make smoke   # confirma que todo sigue funcionando
```

Las migraciones se aplican solas al arrancar cada servicio

(`KUBO\_RUN\_MIGRATIONS=true`): Flyway en IAM, `db:prepare` en Rails y

`ecto.migrate` en Phoenix. Una migración aplicada **nunca se edita**; se crea una nueva.

5. Respaldos

Automatizados con `make backup` (`kubo-infra/scripts/backup.sh`):

| Qué | Cómo | Frecuencia | | -- | -- | -- | | PostgreSQL | `pg\_dump -Fc` por base (`kubo\_iam`, `kubo\_crm`, `kubo\_erp`) | Diaria | | MongoDB | `mongodump --archive` de `kubo\_analytics` | Diaria | | Configuración | `kubo-infra/.env` cifrado con `age` (si hay destinatario) o `chmod 600` | Diaria | | Retención | `KUBO\_BACKUP\_RETENTION\_DAYS` (14 por defecto) | Automática |

```
make backup      # deja un directorio con fecha en kubo-infra/backups/
make restore-drill # restaura en bases kubo_drill_* y compara filas
```

Programación diaria con systemd (copiar a `/etc/systemd/system/`):

```
sudo cp kubo-infra/systemd/kubo-backup.{service,timer} /etc/systemd/system/
sudo systemctl enable --now kubo-backup.timer    # 03:30 con retraso aleatorio
```

Objetivos: RPO 24 h · RTO 4 h. **Un respaldo que nunca se restauró no es un respaldo**: `make restore-drill` toma un respaldo nuevo, lo restaura en bases de prueba, compara el número de filas de las tablas clave y mide el tiempo (en este equipo: 14/14 en 6 segundos). Ejecútalo mensualmente y tras cada cambio de esquema.

****Importante****: MongoDB guarda una ****proyección**** reconstruible desde los eventos del ERP. PostgreSQL es la fuente de verdad y es lo que no puede perderse.

6. Diagnóstico rápido

| Síntoma | Primera revisión | | --- | --- | | La PWA carga pero no hay datos | `make ps` — ¿el gateway está ***healthy***? | | `401` en todas las llamadas | El gateway no puede leer el JWKS: revisa `kubo-iam` | | `502 UPSTREAM\_UNAVAILABLE` | El microservicio destino está caído: `docker logs kubo-erp` | | El tablero no refleja ventas | `docker logs kubo-analytics` — ¿el consumidor está conectado? | | `429` en operación normal | Ajusta `KUBO\_RATE\_LIMIT\_PER\_MINUTE` en el gateway | | El inventario no cuadra | Compara `products.stock` con la suma del kardex (consulta en `02-modelo-datos.md`) |

7. Escalado

El sistema está diseñado para crecer sin reescribir:

1. ****Más memoria/CPU****: subir los `mem\_limit` del compose.
2. ****Más carga en la caja****: escalar el ERP horizontalmente (`docker compose up -d --scale kubo-erp=3`) porque es sin estado; el gateway reparte.
3. ****Bases separadas****: mover cada base a su propia instancia cambiando solo la URL de conexión.
4. ****Clúster****: los mismos artefactos corren en Kubernetes; falta mTLS entre servicios (Fase 5, P-28) y la activación de RLS (Fase 1, P-02).

8. Lo que falta para producción real

| Pendiente | ID · Fase | Impacto | | --- | --- | --- | | Testcontainers en los servicios restantes | P-08 · Fase 2 | IAM y analítica ya la tienen; faltan CRM, ERP y gateway | | Compras y proveedores · sesiones de caja | P-15 · P-16 · Fase 3 | El inventario solo entra por ajuste manual y no hay control de efectivo por turno | | Usuarios y roles en la interfaz · importación de datos | P-20 · P-24 · Fase 3 | El dueño no puede crear vendedores ni cargar su catálogo sin ayuda técnica | | Facturación electrónica DIAN | P-18 · Fase 4 | Requisito legal para facturar | | mTLS entre servicios | P-28 · Fase 5 | Movimiento lateral dentro del clúster |

> Las fases 1 y 2 (confiabilidad y calidad) están cerradas y verificadas; ver

> [`11-plan-de-cierre.md`](11-plan-de-cierre.md).

06 — Manual de usuario

Guía para el dueño del negocio y sus vendedores. No requiere conocimientos

técnicos.

1. Ingresar

1. Abre el navegador en la dirección que te entregó el instalador

(por ejemplo `http://192.168.1.50:3000`).

2. Escribe tu correo y contraseña.

3. Presiona **Ingresar**.

Instala Kubo como aplicación (recomendado): en el navegador busca el ícono de instalar (o el menú : → **Instalar aplicación**). Queda un acceso directo como cualquier otro programa y abre a pantalla completa.

> La primera vez recibirás el usuario administrador. Cámbialo apenas puedas.

2. El tablero

Es la primera pantalla y responde una pregunta: **¿cómo va el negocio hoy?**

| Indicador | Qué significa | | --- | --- | | **Ventas de hoy** | Cuánto se vendió hoy y cuántas transacciones | | **Ingreso acumulado** | Total vendido desde que usas Kubo | | **Ticket promedio** | Lo que gasta un cliente en promedio | | **Unidades vendidas** | Cuántos productos salieron | | **Ventas de los últimos 14 días** | La tendencia: ¿sube o baja? | | **Medios de pago** | Cuánto entra en efectivo, tarjeta o transferencia | | **Productos más vendidos** | En qué se está yendo la mercancía | | **Últimas ventas** | El detalle reciente, venta por venta |

Si el local se queda sin internet, verás la etiqueta **Sin internet**: el

tablero muestra lo último que se sincronizó y puedes seguir vendiendo.

3. Vender (punto de venta)

1. Entra a **Vender**.

2. Toca los productos para agregarlos al carrito (usa el buscador si tienes muchos).

3. Ajusta cantidades con **+** y **-**.

4. Elige el **cliente** (o deja **Consumidor final**).

5. Elige el **medio de pago**: efectivo, tarjeta, transferencia o crédito.

6. Presiona **Cobrar**.

Lo que ocurre al cobrar:

- Se descuenta el inventario en el mismo instante.
- Se asigna un número de venta (V-000001, V-000002...).
- El tablero se actualiza solo, en segundos.

Vender sin internet

Si se cae la conexión, Kubo **no te deja tirado**: la venta se guarda en el navegador y aparece la etiqueta **«1 venta por sincronizar»**. Cuando vuelva la conexión:

- La aplicación la envía automáticamente, o
 - Presiona **Sincronizar** en la barra superior.
- > Mientras no haya conexión no podrás consultar el stock más reciente de otros
> equipos, pero las ventas nunca se pierden.

4. Productos e inventario

En **Productos** ves el catálogo con precio, IVA y existencias.

- **Nuevo producto**: pide código (SKU), nombre, precio de venta, costo, tarifa de IVA y stock mínimo.
- El **precio de venta** es el precio final que paga el cliente (con IVA incluido). Kubo desagrega el impuesto al facturar.
- El **stock mínimo** dispara la alerta de reposición.
- **Ajustar inventario** (ícono de cajas): registra entradas por compra, salidas por merma o conteos físicos.

| Tipo | Cuándo usarlo |

| --- | --- |

| **Entrada (compra)** | Llegó mercancía del proveedor |

| ****Salida (merma, consumo)**** | Se dañó, se venció o se usó internamente |

| ****Fijar stock exacto**** | Después de un conteo físico |

Escribe siempre el ****motivo****: queda en el kardex con fecha y usuario.

- ****Archivar**** (papelera): el producto deja de venderse pero su historia se conserva. Nunca se borra nada de verdad.

Cuando algo está en o por debajo del mínimo, aparece un aviso amarillo arriba:
es el momento de programar una compra.

5. Clientes

En ****Clientes**** está tu cartera.

- ****Nuevo cliente****: nombre o razón social, documento (NIT o cédula), teléfono, correo, ciudad, dirección, etapa y cupo de crédito.

- ****Etapas****: *Prospecto nuevo* → *En negociación* → *Cliente*. Sirve para saber a quién hay que llamar.

- El ****documento y el teléfono se guardan cifrados****: si alguien se lleva una copia de la base de datos, no puede leerlos.

- En la lista solo se ven los últimos dígitos (*******432`**); el dato completo aparece al abrir el cliente.

> ****Protección de datos****: Kubo guarda los datos personales únicamente para

> gestionar la relación comercial. Informa a tus clientes qué guardas y para

> qué, y atiende sus solicitudes de corrección o eliminación.

6. Preguntas frecuentes

****¿Necesito internet para vender?****

No. Solo para sincronizar; las ventas se guardan localmente.

****¿Puedo tener varias cajas al mismo tiempo?****

Sí. Cada equipo entra con su propio usuario. El stock se descuenta en el

servidor, así que dos cajas no pueden vender la misma unidad.

****¿Dónde están mis datos?****

En el equipo donde se instaló Kubo, no en la nube de terceros.

****¿Cuánto cuesta?****

El software es tuyo (licencia MIT). Solo pagas el equipo donde corre.

****Me equivoqué en una venta.****

Pídele al administrador que la ****anule****: el inventario vuelve y queda registrada la anulación (no se borra la historia).

****¿Puedo usar mi lector de código de barras?****

Sí, si escribes el código como texto. Escribe o escanea en el buscador del POS.

****Olvidé mi contraseña.****

En la pantalla de ingreso pulsa ****«¿Olvidaste tu contraseña?»**** y escribe tu correo: recibirás un enlace para crear una nueva. El enlace vence en 30 minutos y solo sirve una vez. Si el correo del servidor aún no está configurado, pídele al administrador que la restablezca.

7. Buenas prácticas

1. ****Registra el costo**** de cada producto: es lo que permite saber tu margen.
2. ****Haz un conteo físico**** una vez al mes y ajusta el stock con el motivo ***«Conteo físico»***: así el kardex cuenta la historia real.
3. ****Usa las etapas de cliente****: es tu lista de trabajo comercial.
4. ****Revisa el tablero cada mañana****: cinco segundos para saber cómo arrancó el día.
5. ****No compartas el usuario administrador****: crea un usuario por vendedor.

07 — Pruebas

La verificación del sistema tiene tres niveles: pruebas unitarias dentro de cada servicio, una prueba de humo end-to-end y verificaciones manuales de operación.

1. Prueba de humo end-to-end (la principal)

make smoke

Ejecuta **87 comprobaciones** contra el sistema en ejecución, usando el API

Gateway como un cliente real. Resultado esperado: `87 pruebas exitosas, 0 fallidas`.

| Bloque | Qué verifica | Comprobaciones | | -- | -- | -- | | 1. Salud | Los cinco servicios de aplicación responden `UP` | 5 | | 2. Autenticación y cookie | Login; **RS256**; negocio en el token; **JWKS**; el refresh **no** viaja en el cuerpo; la cookie es **httpOnly + SameSite=Strict**; rotación; el token refrescado autentica; **reuso** → `401`; **el reuso revoca la familia completa**; logout 204 y borra la cookie | 14 | | 3. Acceso, auditoría y bloqueo | Sin token → `401`; token falsificado → `401`; cabecera inyectada → **ignorada**; **5 intentos fallidos bloquean la cuenta**; **cadena intacta**; entradas verificadas; **versión vigente del hash**; `LOGIN\_FAILED`, `ACCOUNT\_LOCKED` y `REFRESH\_REUSE\_DETECTED` en la bitácora | 11 | | 4. Clientes, cifrado y RLS | Crear cliente; detalle completo; listado **enmascarado**; texto **cifrado**; índice ciego; **búsqueda por documento**; **RLS: sin contexto 0 filas, con contexto > 0** | 8 | | 5. Inventario, venta y outbox | Producto; entrada de 10; venta por `35700.00`; IVA `5700.00`; stock 7; kardex 2; sobreventa → `409`; **el evento queda en la bandeja**; **se publica**; **RLS en ERP** (0 y > 0); **paginación con total real y tope de página** | 14 | | 6. Evento y tablero | Proyección en analítica; evento en MongoDB; tablero; **vista compuesta con las 7 vistas**; sin vistas caídas; zona horaria; **outbox sin fallidos** | 7 | | 7. Anulación | La venta queda `VOIDED`; el inventario vuelve a 10 | 2 | | 8. Aislamiento | Un segundo negocio no ve clientes ni catálogo; **RLS en IAM** (0 filas sin contexto) | 4 | | 9. Recuperación de contraseña | Solicitud 204; enlace en el buzón; consumo 204; **la cuenta se desbloquea con la clave nueva**; el enlace **no se reutiliza**; sin enumeración de usuarios | 6 | | 10. Compras y proveedores | Proveedor creado; compra por `25000.00`; IVA `3991.60`; **suma 5 unidades**; **el costo se actualiza sin IVA**; kardex `PURCHASE`; **anulación revierte el stock**; **RLS en compras** (0 filas sin contexto) | 9 | | 11. Tasa, TLS y trazas | **Límite por usuario** activo; HTTPS 200; HTTP → HTTPS (308); HSTS; **collector de trazas arriba y recibiendo spans** | 6 |

Además del humo, la Fase 2 agregó: **10 contratos OpenAPI** (`make contracts`),

4 pruebas de navegador con axe (`make e2e`), la **carga a 50 cajas**

(`make load`, p95 < 300 ms), los simulacros de bus y restauración, y el gate

`make ci` que los reúne.

La prueba es **idempotente**: crea sus propios datos con marcas de tiempo y puede

ejecutarse tantas veces como haga falta.

2. Pruebas unitarias por servicio

```
# kubo-iam – tokens, reglas de identidad y integración real (31 pruebas)
# Incluye Testcontainers: PostgreSQL real, migraciones Flyway y RLS.
cd kubo-iam && mvn verify

# kubo-gateway – tabla de rutas y rutas públicas (3 pruebas)
cd kubo-gateway && npm test

# kubo-crm – cifrado de campos (8 pruebas, sin base de datos)
cd kubo-crm && ruby test/field_cipher_test.rb

# kubo-erp – aritmética de dinero y outbox (7 pruebas, sin base de datos)
docker run --rm -m 3g -e MIX_ENV=test \
  -v "$PWD/kubo-erp/test:/app/test:ro" --entrypoint bash kubo-kubo-erp \
  -c "cd /app && mix compile >/dev/null 2>&1 && ERL_LIBS=/app/_build/test/lib \
    elixir -e 'ExUnit.start(); Code.require_file(\"test/kubo_erp/sales_totals_test.exs\" \
    Code.require_file(\"test/kubo_erp/outbox_test.exs\")'"

# kubo-analytics – conversión de eventos (6 pruebas, sin MongoDB)
cd kubo-analytics && pip install -r requirements-dev.txt && pytest -q
```

****Total: 55 pruebas**** (unitarias y de integración) más 87 comprobaciones

end-to-end, 10 contratos y 4 pruebas de navegador. La cobertura de dominio se

vigila en CI: IAM $\geq 80\%$ (JaCoCo) y analítica $\geq 80\%$ en su módulo de

procesamiento (pytest-cov).

Cómo ejecutar cada suite

```
make smoke                # 87 comprobaciones end-to-end
make bus-drill             # 4 comprobaciones: caída del bus
make restore-drill         # 14 comprobaciones: restauracion

cd kubo-iam                && mvn test                # 7 pruebas
cd kubo-gateway            && npm test                # 3 pruebas
cd kubo-crm                && ruby test/field_cipher_test.rb # 8 pruebas
cd kubo-analytics          && pytest -q              # 6 pruebas
```

Las 7 pruebas de `kubo-erp` son puras (aritmética decimal y outbox) y ****no pueden**

ejecutarse dentro del contenedor de producción******: al compilar el entorno de

pruebas el contenedor se queda sin memoria (límite de 512 MB), y además la imagen

de ejecución no incluye `test/`. Se ejecutan con más memoria y el directorio de

pruebas montado (comando completo arriba), o en CI con 3 GB.

Qué cubren las pruebas unitarias

| Servicio | Casos | | --- | --- | | IAM | El token se firma con RS256 y tiene 3 segmentos; el JWKS ****no**** expone la llave privada; los refresh tokens son aleatorios y su hash es SHA-256 hex | | Gateway | La tabla cubre los cuatro microservicios; los prefijos anidados resuelven al servicio correcto; una ruta protegida no se confunde con una pública | | CRM | Ida y vuelta del cifrado; dos cifrados del mismo valor difieren (IV aleatorio); ****GCM detecta manipulación****; el índice ciego es determinista y normalizado; enmascarado correcto; valores vacíos no se cifran | | ERP | IVA desagregado de un precio con impuesto incluido; suma de varias líneas; producto exento; el total siempre es subtotal + impuesto | | Analítica | Conversión de importes y cantidades desde el evento; tolerancia a entradas inválidas; normalización del detalle; fechas a UTC |

3. Verificación manual de la interfaz

| # | Paso | Resultado esperado | | --- | --- | | 1 | Abrir `http://localhost:3000` | Pantalla de ingreso con la marca Kubo | | 2 | Ingresar con `admin@kubo.local` / `Admin123!` | Tablero con indicadores | | 3 | Instalar la aplicación desde el navegador | Se abre a pantalla completa, sin barra del navegador | | 4 | Registrar una venta en el POS | Aparece el número de venta y el tablero se actualiza | | 5 | Activar el modo avión y vender | Etiqueta «Sin internet» y la venta queda en cola | | 6 | Recuperar la conexión | La venta se sincroniza y el contador desaparece | | 7 | Abrir un cliente | Documento completo; en el listado, enmascarado | | 8 | Registrar una entrada de inventario | El stock sube y el movimiento queda en el kardex | | 9 | Intentar vender más unidades que el stock | Mensaje «Stock insuficiente» con la cantidad disponible | | 10 | En el ingreso, «¿Olvidaste tu contraseña?» → pedir el enlace | El correo queda en el buzón de demostración (`mail\_outbox`) y el enlace de `/recuperar?token=...` permite elegir una contraseña nueva |

4. Verificación de seguridad

```
# Sin token
curl -o /dev/null -w '%{http_code}\n' http://localhost:9080/api/v1/customers # 401

# Suplantación por cabecera
curl -o /dev/null -w '%{http_code}\n' http://localhost:9080/api/v1/customers \
  -H 'X-User-Id: 00000000-0000-0000-0000-000000000000' # 401

# El texto cifrado no contiene el documento
docker exec kubo-postgres psql -U kubo_root -d kubo_crm -c \
  "select document_number_encrypted from customers limit 1"

# La llave privada no se publica
curl -s http://localhost:9080/api/v1/auth/.well-known/jwks.json | grep -c '"d"' # 0

# Límite de tasa en autenticación
for i in $(seq 1 50); do
  curl -s -o /dev/null -w '%{http_code} ' -X POST http://localhost:9080/api/v1/auth/login \
    -H 'Content-Type: application/json' -d '{"email":"x@x.co","password":"malo"}'
done; echo # inc
```

5. Rendimiento observado

Medido en un equipo de 16 núcleos, a través del gateway (la latencia que percibe el usuario):

| Operación | Tiempo | | --- | --- | | Login (BCrypt + firma RSA) | ~75 ms | | Registrar venta (transacción + kardex) | ~45 ms | | Consultar catálogo (300 productos) | ~6 ms | | **Tablero completo (vista compuesta BFF)** | **~25 ms** (antes 108 ms en 7 viajes) | | Búsqueda por texto con 50.000 registros (ERP) | ~5.7 ms (antes 46 ms) | | Búsqueda por texto con 50.000 registros (CRM) | ~4.4 ms (antes 28 ms) | | Propagación del evento a MongoDB | < 1 s | | Arranque completo del sistema | ~50 s | | Consumo en reposo | ~930 MB |

El detalle de las mediciones, los planes de ejecución y su análisis están en

[`10-auditoria.md`](10-auditoria.md).

6. Pruebas pendientes (fases 1 y 2)

| Prueba | ID · Fase | Por qué falta | | --- | --- | | Integración con base de datos real por servicio | P-08 · Fase 2 | IAM ya la tiene (Testcontainers + RLS); falta replicarla en analítica y los demás servicios | | Contratos (Pact/OpenAPI) automatizados | P-09 · Fase 2 | Hoy el contrato se verifica en la prueba de humo | | Carga (k6): 50 cajas simultáneas | P-10 · Fase 2 | Falta escenario de estrés | | E2E de navegador (Playwright) | P-10 · Fase 2 | Hoy la interfaz se verifica manualmente | | Escaneo

SAST/SCA y de secretos en CI | P-06 · Fase 2 | Las verificaciones son manuales | | Accesibilidad automatizada (axe) | P-26 · Fase 2 | Solo revisión manual de contraste y foco |

08 — Matriz de trazabilidad

Cada requisito del enunciado, cómo se implementó y **dónde se comprueba**. Si un requisito no tiene evidencia verificable, aparece marcado como pendiente.

1. Requisitos del proyecto

#	Requisito	Implementación	Evidencia	Estado
R1	ERP + CRM para PYMES sin acceso a tecnología, sin pagar SaaS	Instalación de un comando en el local, licencia MIT, sin dependencias de nube	<code>README.md</code> · <code>05-despliegue.md</code> · <code>make up</code>	Cumplido
R2	Mínimo 4 microservicios	5 servicios: IAM, CRM, ERP, Analítica y Documentos/Notificaciones (Fase 4, P-19 y P-25)	<code>docker compose ps</code> (12 contenedores) · <code>01-arquitectura.md</code>	Cumplido (4 desplegados)
R3	Mínimo 3 tecnologías distintas	Java, Ruby, Elixir, Python, TypeScript (5)	<code>01-arquitectura.md</code> · ADR-0004	Cumplido
R4	Front dinámico y actual, nivel senior	React 19 + Vite 8 + Tailwind 4, design system con tokens, TanStack Query, gráficas, accesibilidad	<code>kubo-web/</code> · <code>kubo-web/src/components/ui.tsx</code>	Cumplido
R5	Conexión con PostgreSQL y MongoDB	PostgreSQL 17 (IAM, CRM, ERP) + MongoDB 8 (analítica)	<code>docker compose ps</code> · <code>02-modelo-datos.md</code>	Cumplido
R6	Interfaz UI/UX	5 pantallas, estados de carga/vacío/error, toasts, modales accesibles, español, modo offline visible	<code>06-manual-usuario.md</code> · verificación manual en <code>07-pruebas.md</code>	Cumplido
R7	API Gateway	<code>kubo-gateway</code> (NestJS): validación JWT, límites de tasa, correlación, enrutamiento	<code>kubo-gateway/README.md</code> · <code>make smoke</code> bloque 3	Cumplido
R8	Cada microservicio con su base de datos	<code>database-per-service</code> : <code>kubo_iam</code> , <code>kubo_crm</code> , <code>kubo_erp</code> con roles aislados; <code>kubo_analytics</code> en MongoDB	<code>kubo-infra/scripts/init-db.sh</code> · ADR-0003	Cumplido
R9	Repositorio separado por parte del software	9 repositorios: 5 componentes + infra + docs + web + raíz	<code>ls -d kubo-*/</code> · ADR-0002	Cumplido
R10	Documentación detallada del funcionamiento	11 documentos + 11 ADRs + README por servicio + OpenAPI ejecutable + guion de video	<code>kubo-docs/</code> · <code>Kubo-Documentacion.pdf</code>	Cumplido
R11	Encriptado de datos	AES-256-GCM campo a campo + índice ciego HMAC + BCrypt + SHA-256 de refresh tokens + auditoría con cadena de hash versionada + RLS en las tres bases + outbox transaccional	<code>make smoke</code> bloques 3, 4 y 5 · <code>04-seguridad.md</code>	Cumplido
R12	PWA	Service worker, manifest, instalable, cola offline en IndexedDB, sincronización automática	<code>curl localhost:3000/manifest.webmanifest</code> · <code>kubo-web/vite.config.ts</code>	Cumplido
R13	JWT	RS256 con JWKS, access 15 min, refresh rotativo con detección de reuso y <code>cookie httpOnly</code> (BFF)	<code>make smoke</code> bloque 2 · ADR-0007 · ADR-0009 (outbox)	Cumplido
R14	Arquitectura idónea al tipo de proyecto	Microservicios con monolito modular, polyrepo, eventos, multi-tenancy híbrido	<code>01-arquitectura.md</code> · ADR-0001 a 0008	Cumplido
R15	Diseño y seguridad del proyecto	Modelo de confianza, STRIDE por servicio, OWASP, Ley 1581, límites de tasa, anti-suplantación	<code>04-seguridad.md</code> · <code>make smoke</code> bloque 3 y 8	Cumplido

2. Requisitos funcionales del dominio

| # | Requisito | Implementación | Evidencia | | --- | --- | --- | --- | | RF-01 | Autenticar usuarios y aislar negocios | `kubo-iam` + `tenant\_id` en el token | `make smoke` bloques 2 y 8 | | RF-02 | Gestionar clientes con datos personales protegidos | `kubo-crm` con cifrado de campo | `make smoke` bloque 4 | | RF-03 | Gestionar catálogo de productos | `kubo-erp` CRUD + borrado lógico | `make smoke` bloque 5 | | RF-04 | Controlar inventario con trazabilidad | Kardex `stock\_movements` + proyección de stock | `make smoke` bloque 5 y 7 | | RF-05 | Registrar ventas con descuento de inventario | Transacción con bloqueo pesimista | `make smoke` bloque 5 | | RF-06 | Anular ventas devolviendo el inventario | `POST /sales/:id/void` | `make smoke` bloque 7 | | RF-07 | Calcular impuestos (IVA) | Desagregación con `Decimal` en el ERP | `make smoke` bloque 5 | | RF-08 | Tablero de indicadores del negocio | `kubo-analytics` sobre MongoDB | `make smoke` bloque 6 | | RF-09 | Operar sin conexión | Cola IndexedDB + sincronización | Verificación manual `07-pruebas.md` §3 | | RF-10 | Auditar las acciones sensibles | `audit\_logs` con cadena de hash | `GET /api/v1/audit` | | RF-11 | Gestionar proveedores y compras (Fase 3) | `kubo-erp`: compra suma inventario, kardex `PURCHASE` y costo sin IVA | `make smoke` bloque 10 · `make contracts` |

3. Requisitos no funcionales

| # | Atributo | Objetivo | Cómo se sostiene | Evidencia | | --- | --- | --- | --- | | RNF-01 | Rendimiento | Venta < 100 ms | Transacción única + Elixir | `07-pruebas.md` §5 | | RNF-02 | Disponibilidad | El bus caído no bloquea la caja | Publicación asíncrona tolerante a fallos | ADR-0005 | | RNF-03 | Escalabilidad | Escalar la caja sin reescribir | Servicios sin estado | `05-despliegue.md` §7 | | RNF-04 | Seguridad | Sin datos personales legibles en la base | AES-256-GCM + índice ciego | `make smoke` bloque 4 | | RNF-05 | Mantenibilidad | Un lenguaje por contexto con arquitectura limpia | ADR-0004 · README por servicio | | RNF-06 | Portabilidad | Mismo artefacto en local y nube | Docker Compose | `05-despliegue.md` | | RNF-07 | Consumo | < 2 GB de RAM en reposo | Límites por contenedor | `docker stats --no-stream` | | RNF-08 | Usabilidad | Dueño sin conocimientos técnicos | Manual + modo simple + español | `06-manual-usuario.md` | | RNF-09 | Observabilidad | Estado de cada servicio y su dependencia | Sondas `/health` | `make smoke` bloque 1 | | RNF-10 | Trazabilidad | Reconstruir una operación completa | `X-Correlation-Id` propagado | Logs del gateway |

4. Decisiones de arquitectura

| ADR | Decisión | Estado | | --- | --- | --- | | 0001 | Microservicios con monolito modular interno | Aceptada | | 0002 | Polyrepo con contratos como fuente de verdad | Aceptada | | 0003 | Multi-tenancy híbrido (`tenant\_id` + RLS activo) | Aceptada | | 0004 | Poliglotismo deliberado: un lenguaje por contexto | Aceptada | | 0005 | Eventos con RabbitMQ y publicador tolerante a fallos | Aceptada (outbox diferido) | | 0006 | Cifrado de campos personales con AES-256-GCM | Aceptada | | 0007 | JWT RS256 con JWKS en el gateway | Aceptada | | 0008 | Alcance declarado del MVP y recortes conscientes | Aceptada | | 0009 | Outbox transaccional para la entrega de eventos | Aceptada (Fase

1) | 0010 | RLS activo con interceptor de transacción | Aceptada (Fase 1) | 0011 | BFF como capa formal del gateway | Aceptada (Fase 2) |

ADR planificadas: **0012** zona horaria por negocio (Fase 4). Ver

11-plan-de-cierre.md.

5. Requisitos no cubiertos (declarados)

> Los recortes de las fases 1 y 2 (*outbox*, RLS, cookie `httpOnly`,

> recuperación de contraseña, TLS, CI, observabilidad, contratos, carga, E2E y

> accesibilidad) quedaron **implementados y verificados**; ver

> 11-plan-de-cierre.md. Los que siguen:

| Requisito | Motivo | Ruta | | --- | --- | --- | | Testcontainers en CRM, ERP y gateway | IAM y analítica ya los tienen | Fase 2 (P-08, parcial) | | Importación de datos (Excel/CSV, ruta Dolibarr) | El MVP no la incluye: el catálogo se carga por API | Fase 3 (P-24) | | Facturación electrónica DIAN (UBL 2.1, CUFE, QR) | Requiere ser Proveedor Tecnológico autorizado | Fase 4 (P-18) | | Multi-rubro (Vertical Packs) | El MVP demuestra un vertical completo | Fase 4 (P-17) | | Servicio de Documentos (adjuntos y plantillas) | El MVP despliega cuatro servicios; el quinto llega con las notificaciones | Fase 4 (P-25) | | App móvil nativa | La PWA cubre el caso de uso | Fase 5 | | Kubernetes | El despliegue objetivo es un local con Docker Compose | Fase 5 |

> Ningún requisito se declara cumplido sin evidencia verificable. Los recortes

> están documentados en el ADR-0008 y en esta matriz.

09 — Guion de demostración (video)

****Duración objetivo: 6 a 7 minutos.**** El guion está escrito para grabarse en una

sola toma, sin edición, con el sistema ya levantado.

Preparación (antes de grabar)

```
cd "Documents/Proyectos de Programación/Kubo"
make foreign-stop      # libera RAM (detiene Dolibarr, Guacamole, Open Notebook)
make up                # levanta los 12 contenedores
make seed              # carga catálogo, clientes y ventas de ejemplo
make smoke             # verifica 87 comprobaciones end-to-end
```

Deja abiertas estas ventanas:

1. Terminal en la carpeta del proyecto.
2. Navegador en `http://localhost:3000` (sin sesión iniciada).
3. Terminal secundaria para consultar PostgreSQL.
4. Explorador de archivos con la carpeta `Kubo/`.

1. Presentación (0:00 – 0:45)

- > "Hola, soy [nombre], estudiante del Tecnólogo en Análisis y Desarrollo de
- > Software. Este es **Kubo**, un ERP + CRM para pequeñas y medianas empresas que
- > no tienen acceso a tecnología y no quieren pagar suscripciones mensuales.
- > Se instala en el propio local, en un computador pequeño o en un servidor
- > económico, y funciona incluso sin internet."

Muestra la terminal:

```
make up
```

- > "Un solo comando levanta todo el sistema: cuatro microservicios, un API
- > Gateway, la aplicación web y cuatro servicios de datos. Cada microservicio
- > está escrito en un lenguaje distinto y tiene su propia base de datos."

2. Arquitectura (0:45 – 1:45)

Muestra `kubo-docs/01-arquitectura.md` (o el diagrama renderizado).

- > "El navegador solo habla con el API Gateway. El gateway valida el token JWT
- > contra el JWKS del servicio de identidad y enruta cada petición.
- >
- > - **kubo-iam** en Java con Spring Boot: identidad, roles y auditoría.
- > - **kubo-crm** en Ruby on Rails: clientes, con los datos personales cifrados.
- > - **kubo-erp** en Elixir con Phoenix: catálogo, inventario y ventas.

> - **kubo-analytics** en Python con FastAPI: el tablero, sobre MongoDB.

>

> El ERP publica un evento cuando se registra una venta, y analítica lo consume

> por RabbitMQ. La caja nunca espera al tablero."

Muestra la carpeta raíz con los 9 repositorios:

```
ls -d kubo-*/
```

> "Nueve repositorios: uno por cada parte del software, más la infraestructura y

> la documentación. El acceso por HTTPS lo termina Caddy, y el token de sesión

> vive en una cookie que el navegador no puede leer."

3. Inicio de sesión (1:45 – 2:15)

En el navegador, en `http://localhost:3000`:

1. Muestra la pantalla de ingreso con la marca.

2. Ingresa con `admin@kubo.local` / `Admin123!`.

3. Menciona: **"La contraseña se guarda con BCrypt y el token se firma con RSA."**

4. Tablero (2:15 – 3:00)

> "Este es el tablero del negocio. Todos estos indicadores vienen de MongoDB,

> contruidos a partir de los eventos de venta: ventas de hoy, ingreso

> acumulado, ticket promedio, unidades vendidas, la serie de los últimos 14

> días, los productos más vendidos y los medios de pago."

Señala la tarjeta **Ventas de hoy** y la gráfica.

> "Hace un momento no había nada aquí; estos datos llegaron solos desde el ERP."

5. Clientes y cifrado (3:00 – 4:00)

1. Entra a **Clientes**.
2. Señala que el documento aparece enmascarado: `\*\*\*\*\*636`.
3. Abre un cliente y muestra que el detalle sí trae el documento completo.
4. En la terminal secundaria:

```
docker exec kubo-postgres psql -U kubo_root -d kubo_crm -c \  
"select name, left(document_number_encrypted, 28) as cifrado, left(document_number_bidx
```

> "En la base de datos el documento no se puede leer: está cifrado con
> AES-256-GCM, que además detecta cualquier manipulación. La columna de al lado
> es un índice ciego con HMAC: permite buscar por documento exacto sin
> descifrarlo. En el listado mostramos solo los últimos dígitos."

6. Productos e inventario (4:00 – 4:30)

1. Entra a **Productos**: muestra precio, IVA, stock y la alerta de stock bajo.
2. Usa el botón de inventario en un producto y registra una entrada:
Tipo: **Entrada (compra)**, cantidad `10`, motivo `Compra a proveedor`.
3. Menciona: **"Cada movimiento queda en el kardex con fecha, usuario y motivo."**

7. Venta en el POS (4:30 – 5:15)

1. Entra a **Vender**.
 2. Busca un producto, agrégalo dos veces, agrega otro.
 3. Selecciona el cliente ***Panadería El Trigo*** y el medio de pago ***Efectivo***.
 4. Muestra el total y el IVA incluido desagregado.
 5. Presiona **Cobrar**.
- > "La venta se registró como V-00000X. El inventario se descontó en la misma

> transacción y el evento ya viajó a analítica."

6. Vuelve al **Tablero**: el indicador de hoy ya cambió.

8. Funcionamiento sin internet (5:15 – 5:50)

1. En el navegador, activa el **modo avión** (o desconecta el Wi-Fi).

2. Señala la etiqueta **Sin internet** en la barra superior.

3. Registra otra venta en el POS.

> "Sin conexión la venta no se pierde: queda en una cola local del navegador."

4. Reactiva la conexión.

> "Al volver la conexión, Kubo la envía sola y el tablero se actualiza."

5. Muestra el botón **Sincronizar** o la etiqueta de pendientes desapareciendo.

9. Cierre (5:50 – 6:30)

Muestra la documentación:

```
ls kubo-docs/
```

> "La documentación incluye arquitectura con diagramas C4, modelo de datos,

> contrato de la API, seguridad, despliegue, plan de pruebas, matriz de

> trazabilidad y once decisiones de arquitectura registradas como ADR.

>

> Kubo se instala con un comando, corre en menos de 2 GB de memoria y está

> pensado para que un negocio de barrio digitalice sus ventas sin pagar

> suscripciones. Gracias."

Frases de respaldo (por si algo falla en vivo)

| Situación | Qué decir | | --- | --- | | Un contenedor no responde | "Reviso la sonda de salud del servicio..." y ejecuta `make ps` | | El tablero tarda | "La consistencia es eventual: el evento viaja por RabbitMQ" | | Falla el modo offline | Muestra la cola en `IndexedDB` desde las herramientas del navegador | | Falla la demo por completo | Ejecuta `make smoke` y muestra las 87 comprobaciones en verde |

Evidencia complementaria

```
make smoke 2>&1 | tee evidencia-smoke.txt
docker compose -f kubo-infra/docker-compose.yml ps
docker stats --no-stream
```

10 — Auditoría técnica y plan de mejora

| Campo | Valor | | --- | --- | | Fecha | 2026-09-23 | | Alcance | Los 6 servicios, la base de datos, la infraestructura y la documentación | | Método | Medición sobre el sistema en ejecución, planes de ejecución reales y revisión de código | | Estado de partida | 33/33 comprobaciones end-to-end en verde, 10 contenedores sanos | | Estado de cierre | **44/44** (se añadieron 11 comprobaciones para cubrir las correcciones) | | Rondas | Primera: rendimiento y verdad del dato · Segunda: seguridad y consistencia |

1. Resumen ejecutivo

El sistema **funciona de extremo a extremo y está verificado**, pero una auditoría técnica sería no se conforma con que funcione: busca dónde se romperá al crecer, dónde miente un indicador y qué falta para ser un producto y no una demostración. Se encontraron **17 hallazgos**, todos corregidos y medidos en esta misma sesión, y **31 pendientes** priorizados con su plan. De ellos, **9** quedaron resueltos y verificados en la Fase 1 (P-01, P-02, P-03, P-04, P-05, P-11, P-23, P-27 y P-31; ver 11-plan-de-cierre.md).

Los dos más graves no eran de rendimiento:

1. **El indicador «ventas de hoy» usaba el día UTC**: en Colombia todas las ventas posteriores a las 19:00 se contaban al día siguiente — justo las horas de mayor venta de una tienda de barrio.

2. ****La auditoría de seguridad se borraba a sí misma****: los intentos de acceso fallidos y la detección de robo de token se registraban dentro de la transacción que después se revertía al lanzar la excepción. La bitácora quedaba sin los dos eventos que más importan en una investigación, y la invalidación de la familia de tokens tampoco se guardaba: el sistema detectaba el robo y no hacía nada.

El más silencioso, descubierto al construir la vista compuesta, fue otro error de dato: la tarjeta de cartera de clientes del tablero ****mostraba ceros siempre****, porque el CRM devolvía `stats` sin el envoltorio `data` que el resto de la API sí usa. Nada fallaba, nada avisaba: simplemente el dueño veía un dato falso.

Correcciones aplicadas (con medición)

| ID | Hallazgo | Severidad | Medición antes → después | | -- | -- | -- | -- | | A-01 | El día comercial se calculaba en UTC | ****Crítica**** | Ventas de 19:00–23:59 caían en el día siguiente | | A-02 | Búsqueda por texto sin índice (recorría todo) | Alta | ERP 46 ms → ****5.7 ms**** (8×) · CRM 28 ms → ****4.4 ms**** (6×) | | A-03 | Tablero con 7 viajes de red secuenciales | Alta | 108 ms → ****~25 ms**** (BFF, 4×) | | A-04 | ****La auditoría de fallos se revertía con la transacción**** | ****Alta**** | El intento fallido y el robo de token no quedaban registrados | | A-05 | ****La llave JWT era efímera y el compose no la pasaba**** | ****Alta**** | Cada reinicio invalidaba ****todas**** las sesiones | | A-06 | Cadena de auditoría con condición de carrera | Alta | Dos peticiones podían partir la cadena sin detección | | A-07 | Verificación de la cadena inalcanzable y mal definida | Media | Código muerto → endpoint `/audit/verify` que verifica enlace y contenido | | A-08 | Hash calculado con nanosegundos, columna con microsegundos | Media | La verificación habría fallado siempre | | A-09 | Numeración de ventas con `COUNT(\*)` | Media | $O(n)$ → **** $O(\log n)$ **** por venta | | A-10 | Cartera de clientes siempre en cero | Media | Contrato inconsistente → dato falso visible | | A-11 | Sin red de seguridad en la interfaz | Media | Pantalla en blanco → mensaje accionable | | A-12 | Índice ciego implementado pero sin endpoint | Media | Función inalcanzable → `GET /customers/by-document/:doc` | | A-13 | Listado de usuarios sin paginación | Media | Tabla completa → página acotada a 100 | | A-14 | ****Prueba del ERP con la expectativa mal calculada**** | Baja | Esperaba 41650 donde el resultado correcto es 29750 (nunca se había ejecutado) | | A-15 | Efectos secundarios dentro de actualizadores de estado | Baja | Avisos duplicados en StrictMode | | A-16 | Regla de «stock bajo» duplicada en dos vistas | Baja | Una sola definición en el dominio | | A-17 | Código muerto y nombres engañosos | Baja | `deleted?` sin uso, `low\_stock\_alerts` que devolvía rotación |

2. Método de auditoría

1. ****Medición, no estimación.**** Latencia por endpoint con `curl` (promedio de 5

ejecuciones) a través del gateway, es decir, la latencia que percibe el usuario.

2. **Prueba a escala.** Se cargaron 50.000 productos y 50.000 clientes

sintéticos para que el planificador de PostgreSQL eligiera planes reales, y se compararon con `EXPLAIN (ANALYZE, BUFFERS)`. Los datos de prueba se eliminaron al terminar.

3. **Revisión de código** servicio por servicio buscando condiciones de carrera, rutas sin usar y supuestos implícitos.

4. **Verificación funcional** con la prueba de humo (33 comprobaciones al inicio, 44 al cierre).

5. **Prueba de manipulación**: se editó una fila de la bitácora con SQL para comprobar que la verificación la detecta.

3. Rendimiento medido

3.1 Latencia por endpoint (a través del gateway, red local)

Endpoint	Antes	Después
<code>POST /auth/login</code> (BCrypt + firma RSA)	75 ms	75 ms
<code>GET /products</code>	6 ms	6 ms
<code>GET /customers</code>	9 ms	9 ms
<code>GET /products/stats</code>	8 ms	8 ms
<code>GET /dashboard/*</code> (6 endpoints)	5–6 ms cada uno	—
<code>GET /dashboard/overview</code> (vista compuesta)	108 ms (7 viajes)	~25 ms (rango 11–42 ms)

3.2 Búsqueda por texto con 50.000 registros

```
-- ERP: productos por nombre o SKU
ANTES   Limit → Sort → Index Scan → Filter → Rows Removed by Filter: 49999
        Execution Time: 46.6 ms
DESPUÉS Bitmap Heap Scan → BitmapOr → Bitmap Index Scan (idx_products_name_trgm)
        Execution Time: 5.7 ms      → 8× más rápido

-- CRM: clientes por nombre o correo
ANTES   28 ms (recorría todos los clientes del negocio)
DESPUÉS 4.4 ms con índices en nombre Y correo → 6× más rápido
```

> **Lección de diseño:** con un solo índice (nombre) el CRM seguía lento, porque

> el `OR` con la columna de correo obligaba a descartar el plan indexado. En

> búsquedas con `OR` hay que indexar **todas** las columnas participantes.

3.3 Recursos

| Recurso | Medición | Evaluación | | --- | --- | --- | | RAM total del sistema | ~930 MB | Cabe en un VPS de USD 6/mes | | Servicio más pesado | `kubo-iam` 282 MB (JVM) | Esperable; acotado con `-Xmx256m` | | Resto de servicios | 13–157 MB | Correcto | | Arranque completo | ~50 s | Aceptable; se puede cachear mejor |

4. Hallazgos corregidos (detalle)

A-01 · Día comercial en UTC — ****Crítica****

```
-- Una venta registrada a las 20:00 hora Colombia (01:00 UTC del día siguiente)
momento_utc (columna)      = 2026-09-24 01:00:00
fecha que usaba Kubo        = 2026-09-24    ← incorrecta
fecha real del negocio      = 2026-09-23    ← correcta
```

****Impacto****: el KPI «Ventas de hoy» y la serie diaria del tablero atribuían al día siguiente las ventas de 19:00 a 23:59. En un negocio de barrio esa franja es la de mayor venta: el indicador más visible del producto mentía.

****Corrección****: zona horaria del negocio configurable (`KUBO\_TIMEZONE`, por defecto `America/Bogota`). El almacenamiento sigue en UTC (correcto); el cálculo del día comercial se hace en la zona del negocio:

- ERP: `( ? AT TIME ZONE 'UTC' AT TIME ZONE ? )::date = ?` — la doble conversión es necesaria porque las columnas son `timestamp without time zone`.

- Analítica: `\$dateToString` con `timezone` y la ventana de «hoy» calculada en la zona del negocio y convertida a UTC para consultar.

A-02 · Búsqueda sin índice — Alta

El `ILIKE '%termino%'` del POS y del CRM no puede apoyarse en un índice B-tree. Con catálogos de barrio no se nota; con 50.000 referencias el sistema recorría todo.

Se añadieron índices ****GIN con `pg\_trgm`**** mediante migraciones versionadas (`pg\_trgm` es extensión **trusted** desde PostgreSQL 13: no requiere superusuario).

A-03 · Siete viajes para pintar el tablero — Alta

La PWA pedía 7 endpoints en secuencia. Se implementó el patrón ****BFF (Backend For**

Frontend)**: el gateway expone `GET /api/v1/dashboard/overview` y consulta los servicios **en paralelo** dentro de la red privada, devolviendo una sola respuesta.

- `Promise.allSettled`: si una vista falla, el tablero se entrega **parcial** y

avisa cuál falta, en lugar de mostrar una pantalla de error.

- Tiempo de espera por servicio: 5 s.

- La ruta la atiende el gateway (`GATEWAY\_OWNED\_PATHS`), no se reenvía.

A-04 · La auditoría de seguridad se borraba a sí misma — Alta

El código registraba el intento fallido y el robo de token **dentro de la misma transacción** de la petición:

```
if (!passwordEncoder.matches(...)) {  
    auditService.record(..., "LOGIN_FAILED", ...);    // se inserta...  
    throw DomainException.unauthorized(...);        // ...y el rollback lo borra  
}
```

Los dos eventos que más importan en una investigación forense (intentos de acceso

fallidos y reutilización de un token robado) **desaparecían de la bitácora**. Peor

aún: en el caso de reuso, la revocación de todas las sesiones del usuario ocurría

en esa misma transacción, así que el sistema **detectaba el robo y no cerraba nada**.

Corrección: dos caminos según la semántica del evento.

| Ruta | Transacción | Por qué | | --- | --- | | Éxito (login, registro, rotación) | La de la operación | No debe existir un registro de algo que finalmente no ocurrió | | Fallo (`LOGIN\_FAILED`, `REFRESH\_REUSE\_DETECTED`) | **Independiente** (`REQUIRES\_NEW`) | El registro debe sobrevivir al rollback |

La revocación de la familia de tokens se movió a `SecurityIncidentService`, que abre

su propia transacción: al no existir en ella ningún bloqueo de la cadena, tampoco

puede producirse un bloqueo muto con el `advisory lock` de auditoría.

Verificado en el humo: el intento fallido y el reuso **aparecen en la bitácora**.

A-05 · La llave JWT era efímera — Alta

`TokenService` generaba un par RSA en memoria cuando no había llave configurada, y

el ``docker-compose.yml`` ****nunca pasaba ``KUBO_JWT_PRIVATE_KEY``****. Consecuencia: cada reinicio del servicio de identidad invalidaba ****todas las sesiones**** de todos los usuarios. Se descubrió porque el propio humo falló en cascada tras reconstruir el contenedor.

****Corrección****: el compose propaga la llave y el entorno de demostración usa una llave RSA real generada con ``openssl``, guardada en el ``.env`` (no versionado).

Verificado: el mismo token sigue siendo válido ****después**** de reiniciar el servicio.

Además se redujo la ventana de caché del JWKS en el gateway (30 s → 10 s): si algún día se rota la llave, el gateway reconoce el nuevo ``kid`` casi de inmediato. El tiempo de espera existe para que nadie pueda saturar el servicio de identidad pidiendo el JWKS con ``kid`` inventados.

A-06 · Cadena de auditoría con condición de carrera — Alta

``record()`` leía el último hash y luego insertaba. Dos peticiones concurrentes podían leer el mismo hash anterior y ****partir la cadena****: dos entradas apuntando al mismo predecesor, y la manipulación dejaba de ser detectable.

****Corrección****: ``pg_advisory_xact_lock`` al inicio de la transacción. Serializa la escritura de la cadena sin bloquear ninguna tabla y se libera automáticamente al terminar la transacción.

A-09 · Numeración de ventas con ``COUNT(*)`` — Media

Cada venta contaba todas las ventas del negocio (verificado con ``EXPLAIN``: ``Index Only Scan`` sobre 50.000 filas). Ahora se toma el ****máximo**** del consecutivo, que el índice único ``(tenant_id, number)`` resuelve en tiempo logarítmico. La restricción única sigue protegiendo la concurrencia y el reintento ya existente la resuelve.

A-06 a A-08 · Integridad de la bitácora

| Hallazgo | Corrección | | -- | -- | | Condición de carrera al encadenar hashes (A-06) |
``pg_advisory_xact_lock`` al inicio de la transacción: serializa la escritura sin bloquear tablas | |

Verificación inalcanzable y mal definida (A-07) | Nuevo `GET /audit/verify``: verifica que cada entrada **enlace** con la anterior y que su **hash** corresponda al contenido. Se expone el hash más antiguo de la ventana para poder encadenar verificaciones | Hash con nanosegundos y columna con microsegundos (A-08) | El instante se trunca a microsegundos antes de calcular el hash: de otro modo la verificación fallaría siempre |

Prueba de manipulación (ejecutada): se editó una fila con SQL y la verificación

devolvió `chainIntact: false``. Una edición parcial rompe la verificación; falsificar

la bitácora completa exigiría recalcular toda la cadena.

A-09 a A-17 · Consistencia, contrato, interfaz y limpieza

| Hallazgo | Corrección | | -- | -- | | Numeración de ventas con `COUNT(*)`` (A-09) | Se toma el **máximo** del consecutivo: el índice único lo resuelve en tiempo logarítmico | | `customers/stats`` sin envoltorio `data`` (A-10) | Unificado al contrato del resto de la API. Además el gateway cae al cuerpo completo si un servicio no envuelve: una inconsistencia degrada la forma, no borra una vista | | Un error de render dejaba la pantalla en blanco (A-11) | `ErrorBoundary`` con mensaje entendible, botón de recarga y registro del error | | `Customer.find_by_document`` inalcanzable (A-12) | `GET /api/v1/customers/by-document/:document`` — el caso de uso real del mostrador | | `GET /users`` devolvía la tabla completa (A-13) | Paginación con tamaño acotado en el servidor (máx. 100) | | **Prueba del ERP con expectativa mal calculada** (A-14) | Esperaba `41650`` donde el resultado correcto es `29750``. La prueba **nunca se había ejecutado**: el contenedor se quedaba sin memoria al compilar el entorno de pruebas. Ahora corre en 4/4 | | Efectos secundarios en actualizadores de estado (A-15) | Validaciones movidas fuera: sin avisos duplicados en StrictMode | | Regla de «stock bajo» duplicada (A-16) | Una sola definición en el dominio (`Product.low_stock?/1``) usada por las dos vistas | | Código muerto y nombres engañosos (A-17) | `Customer#deleted?`` eliminado; `low_stock_alerts`` renombrado a `product_rotation`` (devolvía rotación, no alertas) |

Nota de migración: bitácora anterior a la corrección

Los registros de auditoría creados **antes** de A-08 se hashearón con nanosegundos, por lo que no son verificables con el algoritmo corregido. En un sistema en producción esto exigiría **versionar el algoritmo** (una columna `hash_version``) y mantener el verificador anterior para el histórico. Aquí, al ser datos de demostración, la bitácora se reinició para partir de una cadena verificable. Queda como pendiente P-23.

5. Pendientes priorizados

- > **Fase 1 aplicada (2026-09-24).** Resueltos y verificados: P-01 (outbox transaccional), P-02 (RLS activo en las tres bases), P-03 (cookie `httpOnly`), P-04 (recuperación de contraseña), P-05 (respaldos + simulacro), P-11 (bloqueo de cuenta), P-23 (hash de auditoría versionado), P-27 (TLS en la instalación) y P-31 (límite de tasa por usuario). Evidencia: `make smoke` 73/73, `make bus-drill` 4/4, `make restore-drill` 14/14, ADR-0009 y ADR-0010.
- >
- > **Fase 2 aplicada (2026-09-24).** Resueltos y verificados: P-06 (CI por repositorio + gate local `make ci`), P-07 (OpenTelemetry en los cinco servicios con collector propio), P-09 (contratos OpenAPI ejecutables, 8/8), P-10 (carga a 50 cajas con p95 149.76 ms y E2E con Playwright), P-12 (caché por sesión), P-13 (paginación real en productos, ventas y kardex), P-14 (digests de imágenes) y P-26 (axe en CI, con tres defectos de accesibilidad corregidos).
- > Cobertura de dominio: IAM 85.5 % y analítica 91 %, con gates en CI.
- > **P-08 queda parcial**: Testcontainers en IAM; falta replicarlo en analítica y los demás servicios. `make smoke` 87/87, ADR-0011.
- >
- > **Fase 3 en curso (2026-09-24).** P-15 (compras y proveedores) implementado: dominio, API, RLS, evento `purchase.received`, 9 comprobaciones en el humo y pantalla en la PWA. Siguen P-16 (caja), P-20, P-24 y P-21.

5.1 Bloqueantes para un negocio real

| ID | Pendiente | Riesgo | Esfuerzo | | -- | -- | -- | -- | | P-01 | **Transactional outbox** | Un evento se pierde si el proceso muere justo tras el `commit`: el tablero no reflejaría una venta ya cobrada | 1-2 días | | P-02 | **Activar RLS** con interceptor de transacción | Hoy el aislamiento entre negocios depende de que toda consulta filtre por `tenant\_id`. Un olvido no se detecta | 2-3 días | | P-03 | **Refresh token en cookie `httpOnly`** (BFF) | Un XSS podría robar el token de refresco | 2 días | | P-04 | **Recuperación de contraseña** | Hoy solo un administrador puede restablecerla desde la base | 1 día | | P-05 | **Respaldos automatizados + simulacro de restauración** | Documentado pero no

ejecutado: un respaldo sin probar no es un respaldo | 1 día | | P-27 | ****TLS/HTTPS en la instalación**** (Caddy y verificación en el humo) | Sin cifrado en tránsito, las credenciales viajan en claro por la red del local | 1 día |

5.2 Calidad e ingeniería

| ID | Pendiente | Por qué importa | Esfuerzo | | --- | --- | --- | --- | | P-06 | ****CI/CD**** (lint + pruebas + SAST + escaneo de secretos por repo) | Las verificaciones son manuales; un `make smoke` no protege contra una regresión de seguridad | 2–3 días | | P-07 | ****Observabilidad**** (OpenTelemetry → Grafana/Loki/Tempo) | Hoy hay sondas de salud y logs locales: no hay trazas ni métricas históricas | 3 días | | P-08 | ****Pruebas de integración con Testcontainers**** por servicio | Las pruebas actuales son unitarias puras + humo; falta cubrir los repositorios | 3 días | | P-09 | ****Pruebas de contrato**** (OpenAPI ejecutable + Pact) | El contrato se verifica de facto en el humo; un cambio incompatible se detectaría tarde | 2 días | | P-10 | ****E2E de navegador**** (Playwright) y ****carga**** (k6, 50 cajas) | La interfaz se verifica a mano; el comportamiento bajo concurrencia no está medido | 2 días | | P-11 | ****Bloqueo de cuenta**** tras N intentos fallidos | Hoy solo hay límite de tasa por IP | 0.5 día | | P-12 | ****Caché del service worker por usuario**** | En un equipo compartido, el caché de lecturas de un usuario podría servirse a otro si no cierra sesión | 0.5 día | | P-13 | ****Paginación**** en productos, ventas y movimientos | El ERP corta en 300/200 registros; un negocio grande los supera | 1 día | | P-14 | ****Empaquetado de imágenes por digest**** y reservas de recursos | Las etiquetas (`postgres:17`) pueden cambiar; conviene fijar el digest | 0.5 día | | P-23 | ****Versionar el algoritmo de hash de auditoría**** | Un cambio de algoritmo deja el histórico sin verificar (ver nota de migración) | 1 día | | P-26 | ****Accesibilidad automatizada**** (axe en CI) | Hoy solo hay revisión manual de contraste y foco | 0.5 día | | P-30 | ****Segundo factor (TOTP)**** para el propietario | Robo de credenciales del administrador | 1–2 días | | P-31 | ****Límite de tasa por usuario**** (no solo por negocio o IP) | Abuso desde una cuenta comprometida | 0.5 día |

5.3 Alcance funcional (lo que separa el MVP de un producto)

| ID | Módulo | Valor para el negocio | | --- | --- | --- | | P-15 | ****Compras y proveedores**** | Cierra el ciclo del inventario: hoy solo entran mercancías por ajuste manual | | P-16 | ****Sesiones de caja**** (apertura, cierre, arqueo) | Control de efectivo por turno: es lo primero que pide un tendero | | P-17 | ****Vertical Packs**** (retail, servicios, restaurantes, agro) | Convierte Kubo en multi-rubro sin duplicar código (ya diseñado) | | P-18 | ****Facturación electrónica DIAN**** (UBL 2.1, CUFE, QR) | Requisito legal para facturar; exige ser Proveedor Tecnológico | | P-19 | ****Notificaciones**** (WhatsApp / correo) | Cobranza, confirmaciones y avisos de stock: el quinto servicio planificado | | P-20 | ****Usuarios y roles en la interfaz**** | Hoy el API existe pero el dueño no puede crear vendedores sin ayuda técnica | | P-21 | ****Reportes exportables**** (CSV/PDF) y ****comprobante de venta imprimible**** (ticket) | Contabilidad y bancos piden soportes; el mostrador necesita el comprobante | | P-22 | ****Multi-bodega**** y transferencias | Para negocios con más de un punto | | P-24 | ****Importación de datos**** (Excel/CSV y ruta Dolibarr) | Sin esto, digitalizar un catálogo existente es manual: la primera barrera de adopción | | P-25 | ****Servicio de Documentos**** (adjuntos y plantillas) | Completa el quinto servicio planificado junto con las notificaciones |

5.4 Escala (Fase 5, backlog declarado)

| ID | Pendiente | Por qué | | --- | --- | --- | | P-28 | mTLS entre gateway y servicios | Movimiento lateral dentro del clúster | | P-29 | Rotación de claves de cifrado de campo (KEK/DEK) | Compromiso de una clave a largo plazo |

Líneas declaradas sin ID: instalación remota (Terraform/Ansible), multi-tenant SaaS

(zona horaria por negocio y onboarding), app móvil nativa y operador de respaldos.

6. Evaluación arquitectónica

Lo que está bien y no conviene tocar

- ****Fronteras por contexto delimitado****: cada servicio es dueño de su esquema y ningún servicio consulta la base de otro. Es la decisión que más ha simplificado todo lo demás.

- ****Seguridad en el borde****: un único punto de validación de identidad (gateway + JWKS) evita repartir la llave privada y centraliza la política.

- ****Tolerancia a fallos en cadena****: la caja no depende del bus ni de analítica.

Verificado: con RabbitMQ caído la venta se registra igual.

- ****Cifrado de campo con índice ciego****: resuelve la tensión entre proteger el dato y poder buscarlo, sin cifrado determinista (que filtra información).

Lo que hay que cambiar

| Cambio | Motivo | ADR a escribir | Fase | | --- | --- | --- | --- | | Outbox transaccional | Único punto donde el sistema puede perder un dato ya confirmado al usuario | ADR-0009 | Fase 1 | | Interceptor de tenant + RLS activo | Convertir el aislamiento en garantía del motor, no del programador | ADR-0010 | Fase 1 | | BFF como capa formal | Ya existe para el tablero; conviene declararlo patrón del gateway y no un caso puntual | ADR-0011 | Fase 2 | | Zona horaria por negocio en la tabla `tenants` | Hoy es global por instalación; en multi-tenant debe ser un atributo del negocio | ADR-0012 | Fase 4 |

Escalado

| Nivel | Qué hacer | Cuándo | | --- | --- | --- | | 1 negocio | Un VPS, Docker Compose (estado actual) | Ahora | | Varias cajas | Escalar `kubo-erp` (sin estado) y separar PostgreSQL a su instancia | > 5 cajas | | Varios negocios | Activar RLS, zona horaria por tenant, BFF, caché de lecturas | Producto SaaS | | Clúster | k3s + mTLS entre servicios + operador de respaldos | > 50 negocios |

7. Plan de mejora

El plan de cierre completo vive en 11-plan-de-cierre.md.

Resumen:

| Fase | Foco | Duración estimada (30 h/semana) | | -- | -- | -- | | 0 | Consistencia de la documentación | 2–3 h (completada) | | 1 | Confiabilidad: outbox, RLS, cookie, respaldos, TLS | ≈3 semanas | | 2 | Calidad y observabilidad: CI, OTel, pruebas, accesibilidad | ≈4 semanas | | 3 | Núcleo comercial: compras, caja, usuarios, importación, comprobantes | 5–7 semanas | | 4 | Diferenciadores: verticales, DIAN, notificaciones, multi-bodega | 5–6 semanas | | 5 | Escala: k3s + mTLS, SaaS multi-tenant, app móvil | backlog declarado |

Cada fase cierra solo con su criterio medido; el detalle de pasos, IDs y criterios de aceptación está en el documento enlazado.

8. Conclusión

El sistema pasó de **«funciona»** a **«es correcto y sostiene la escala de su mercado»**. La primera ronda arregló la **«verdad del dato»** (un indicador que mentía a partir de las 19:00 y una tarjeta que mostraba ceros sin fallar) y el **«rendimiento»** (búsquedas que recorrían la tabla completa, un tablero que pedía siete viajes). La segunda ronda cerró la **«seguridad»**: la auditoría registraba los eventos de éxito pero borraba los de fallo —justo los que sirven para investigar—, y la detección de robo de token no llegaba a cerrar las sesiones.

Ninguno de esos defectos aparecía en una prueba funcional: el sistema respondía 200, el humo pasaba en verde y la interfaz se veía bien. Los encontró la revisión del **«código y de los datos»**, no la de las respuestas.

Lo que falta no es cosmético: son las piezas que convierten una demostración técnica sólida en un producto que un tendero puede usar todos los días sin sobresaltos — **«garantía de entrega de eventos, aislamiento impuesto por el motor, automatización de calidad y los módulos que el negocio realmente pide»** (compras, caja, usuarios). El plan de las fases 1 a 4 está ordenado por riesgo, no por

lucimiento: primero lo que puede perder o mezclar datos, después lo que evita regresiones, y al final lo que amplía el mercado.

9. Cómo se sostiene este nivel de rigor

La auditoría no fue una lista de buenas intenciones: cada hallazgo se ****midió**** (`EXPLAIN ANALYZE` con 50.000 filas sintéticas, latencia promedio de 5 ejecuciones, `docker stats`) y cada corrección se ****verificó**** con una comprobación añadida al humo. Once de las 44 comprobaciones existen porque un defecto concreto obligó a escribirlas.

Ese es el criterio para presentar el MVP con tranquilidad: no que no tenga fallos —los tenía, diecisiete— sino que ****cada fallo tiene ahora una prueba que lo vigila****.

11 — Plan de cierre

| Campo | Valor | | --- | --- | | Fecha | 2026-09-23 | | Objetivo | Producto comercializable en la región (Fases 0–4); la Fase 5 queda como backlog declarado | | Punto de partida | 44/44 comprobaciones en verde · 10 contenedores sanos · 9 repositorios limpios | | Dedicación | ~30 h/semana (≈ 3.75 jornadas de 8 h) | | Estado | Fases 0–3 completadas · Fase 4 en curso (F4.6 completado) · Fase 5 como backlog declarado |

Este documento ordena los 31 pendientes de 10-auditoria.md en fases con criterio de cierre medible. No sustituye a la auditoría: la usa como catálogo de origen.

1. Decisiones registradas

| Decisión | Elección | | --- | --- | | Objetivo del cierre | Producto comercializable (Fases 0–4; Fase 5 como backlog declarado) | | Primer módulo de la Fase 3 | Compras + Caja | | Dedicación | ~30 h/semana |

2. Hallazgos de la revisión de cierre (R-1...R-9)

La revisión que originó este plan encontró nueve inconsistencias. Todas quedaron corregidas en la Fase 0 o asignadas a una fase con ID.

| ID | Hallazgo | Resolución | Estado | | --- | --- | --- | --- | | R-1 | IDs de hallazgos duplicados en la auditoría (las dos rondas colisionaron: cuatro secciones A-04/A-05) | Renumerados a ****A-06**** y ****A-09**** en §4 | Corregido | | R-2 | La importación de datos (Excel/CSV/Dolibarr) no estaba en ningún plan | ****P-24**** · Fase 3 | Planificado | | R-3 | El quinto servicio figuraba como «planificado» en trazabilidad pero sin fase | ****P-25**** Documentos + ****P-19**** Notificaciones · Fase 4 | Planificado | | R-4 | Faltaban los ADR 0009–0012 que la propia auditoría exige | 0009 y 0010 · Fase 1 · 0011 · Fase 2 · 0012 · Fase 4 | Planificado | | R-5 | La auditoría de accesibilidad (axe) estaba en pruebas pendientes sin ID | ****P-26**** · Fase 2 | Planificado | | R-6 | TLS/HTTPS solo estaba en la guía de despliegue, no en el plan | ****P-27**** · Fase 1 | Planificado | | R-7 | RLS versionada solo en `kubo-iam`, contra lo que afirmaba `04-seguridad.md` §4.3 | Script para CRM y ERP dentro de ****P-02**** · Fase 1 | Planificado | | R-8 | Pendientes de `04-seguridad.md` §10 sin ID ni fase (mTLS, rotación de claves, 2FA, rate limiting por usuario) | ****P-28...P-31**** con fase asignada | Planificado | | R-9 | Conteos y referencias de fase desactualizados (8/9 contenedores, 37 comprobaciones, «16 pendientes») | Corregidos en `01`, `04`, `05`, `07`, `08`, `09` y `10` | Corregido |

3. Fases

Fase 0 — Consistencia (completada, 2–3 h)

| Paso | Acción | Estado | | --- | --- | --- | | F0.1 | Renumerar los A-04/A-05 duplicados (A-06, A-09) | Hecho | | F0.2 | Corregir conteos y la afirmación de RLS | Hecho | | F0.3 | Incorporar P-24...P-31 con ID y fase | Hecho | | F0.4 | Renumerar fases 0–5 en todos los documentos y ADR | Hecho | | F0.5 | Crear este documento, enlazarlo, añadirlo al PDF y commit | Hecho |

****Criterio****: el resumen y el detalle de la auditoría coinciden y el plan es

navegable desde el índice.

Fase 1 — Confiabilidad (≈3 semanas) — bloqueante para un negocio real

| Paso | Pendiente | ID | Esfuerzo | | --- | --- | --- | --- | | F1.1 | Outbox transaccional + publicador de barrido + ADR-0009 | P-01 | 1–2 d | | F1.2 | Interceptor de tenant + RLS en las tres bases (escribir el script en CRM y ERP) + ADR-0010 | P-02 · R-7 | 2–3 d | | F1.3 | Refresh token en cookie `httpOnly` + `SameSite` (BFF) | P-03 | 2 d | | F1.4 | Recuperación de contraseña por correo | P-04 | 1 d | | F1.5 | Respalos automatizados + simulacro cronometrado | P-05 | 1 d | | F1.6 | Bloqueo de cuenta tras N intentos fallidos | P-11 | 0.5 d | | F1.7 | Límite de tasa por usuario | P-31 | 0.5 d | | F1.8 | Versionado del algoritmo de hash de auditoría (`hash\_version`) | P-23 | 1 d | | F1.9 | TLS en la instalación (Caddy + verificación en el humo) | P-27 | 1 d |

****Criterio de aceptación****: prueba de caída del bus sin pérdida de eventos · una

consulta sin filtro de tenant devuelve cero filas · restauración completa

cronometrada por debajo de 4 h · `make smoke` con ≥ 55 comprobaciones en verde.

****Estado: completada (2026-09-24).** Evidencia: `make smoke` **73/73** .**

`make bus-drill` **4/4** (venta con el bus caído, evento `PENDING`, publicación al volver) · `make restore-drill` **14/14** con RTO de 6 s · consulta sin contexto =

****0 filas**** en IAM, CRM y ERP · ADR-0009 y ADR-0010 aceptados.

Dos defectos reales aparecieron al integrar la fase y quedaron corregidos:

1. ****RLS en transacciones `REQUIRES\_NEW`****: el contador de intentos fallidos y la revocación de la familia de tokens corrían en otra conexión sin contexto y actualizaban cero filas en silencio (el sistema detectaba el robo y no cerraba nada). `SecurityIncidentService` ahora fija su propia marca de sistema, y el humo comprueba la revocación completa de la familia.

2. ****Transacciones anidadas de Ecto sin savepoint****: un `Repo.rollback` de negocio (stock insuficiente) abortaba la transacción externa del interceptor y tumbaba la petición después de responder. `Repo.scoped\_transaction/1` usa savepoints.

En el mismo paso se corrigió un contrato roto del ERP: `Catalog.adjust\_stock` devolvía `{ok, {product, movement}}` y el controlador esperaba `{ok, product, movement}` (el ajuste de inventario respondía 500).

Fase 2 — Calidad y observabilidad (≈4 semanas)

| Paso | Pendiente | ID | Esfuerzo | | -- | -- | -- | -- | | F2.1 | CI por repositorio (lint, pruebas, SAST, secretos, SBOM) + ADR-0011 | P-06 | 2–3 d | | F2.2 | OpenTelemetry en los cuatro lenguajes → Grafana/Loki/Tempo | P-07 | 3 d | | F2.3 | Pruebas de integración con Testcontainers | P-08 | 3 d | | F2.4 | Contratos ejecutables (OpenAPI/Pact) | P-09 | 2 d | | F2.5 | E2E (Playwright) + carga (k6, 50 cajas) | P-10 | 2 d | | F2.6 | Caché del service worker por usuario + axe en CI | P-12 · P-26 | 1 d | | F2.7 | Paginación en productos, ventas y movimientos + digests de imagen | P-13 · P-14 | 1.5 d |

****Criterio de aceptación****: una regresión de seguridad o de contrato bloquea el merge · p95 del POS por debajo de 300 ms con 50 cajas · cobertura de dominio ≥ 80 %.

****Estado: completada (2026-09-24).** Evidencia:**

- ****P-06 CI****: `.gitlab-ci.yml` en los ocho repositorios (pruebas, SAST, secretos, dependencias) y gate local `make ci` → ****9/9 en verde**** (secretos, suites,

contratos, humo, E2E).

- **P-07 OpenTelemetry**: collector OTLP propio (`kubo-otel`) y los **cinco** servicios exportando trazas (gateway, IAM, CRM, ERP y analítica), con `make observability` para el stack visual (Tempo + Grafana). El humo verifica que el collector recibe trazas.

- **P-09 contratos**: `kubo-docs/api/openapi.json` + `make contracts` → **10/10 contratos** validados con Ajv contra la API viva.

- **P-10 carga y E2E**: `make load` (50 cajas) → **100 %** de ventas exitosas, p95 = 149.76 ms; Playwright + axe → **4/4**, con tres defectos reales de accesibilidad corregidos (contraste y un `select` sin nombre accesible).

- **P-12 · P-13 · P-14 · P-26**: caché por sesión, paginación real con totales (los listados mentían en `total`), digests de imágenes y axe en CI.

- **Cobertura**: IAM **85.5 %** de dominio (gate JaCoCo ≥ 80 % en `mvn verify`) y analítica **91 %** en su módulo de dominio (gate `--cov-fail-under=80`).

- **P-08 integración con infraestructura real**: en los cinco servicios, contra el motor y no contra dobles. IAM: RLS + cadena de auditoría sobre PostgreSQL (2 pruebas). Analítica: deduplicación y proyección sobre MongoDB efímero (testcontainers, 2 pruebas). ERP: RLS sin contexto = 0 filas, numeración consecutiva por negocio, atomicidad venta + bandeja de salida y kardex con anulación sobre PostgreSQL real (4 pruebas). CRM: aislamiento RLS entre negocios y round-trip del cifrado con índice ciego sobre PostgreSQL real (2 pruebas). Gateway: ventana de límite de tasa, 429 al exceder y cuota independiente por usuario contra Redis real (3 pruebas). En todos, las pruebas corren como el **rol** de la aplicación, no como superusuario: un superusuario ignora RLS incluso con `FORCE` y la prueba dejaría de probar algo. Los runners son `kubo-infra/scripts/erp-tests.sh`, `crm-tests.sh` y

`analytics-tests.sh` (integrados en `make ci`), y cada `.gitlab-ci.yml` añade el servicio correspondiente (postgres/redis).

- ****Defectos reales corregidos al añadir P-08****: el CI de analítica no podía importar `app` (faltaba `pytest.ini` con `pythonpath`) ni tenía instalado `pytest-cov` pese a usar `--cov-fail-under`; el guard de las pruebas de integración exigía el binario `docker` cuando basta el socket; y el volcado de `schema.rb` rompía las migraciones del CRM en el contenedor de pruebas.

Defectos reales corregidos durante la fase (además de los de implementación): numeración de ventas con `ON CONFLICT` que devolvía un id inexistente, RLS con subconsulta inestable bajo concurrencia (denormalizado `tenant\_id` en `sale\_items`), interceptor de tenant con `Repo.rollback` fuera de transacción real, y la cobertura de listados que reportaba el tamaño de página como total.

Fase 3 — Núcleo comercial (5–7 semanas) — orden aprobado: Compras + Caja

| Paso | Pendiente | ID | Esfuerzo | | -- | -- | -- | -- | | F3.1 | Compras y proveedores | P-15 | 5–7 d | | F3.2 | Sesiones de caja (apertura, cierre, arqueo) | P-16 | 5–7 d | | F3.3 | Usuarios y roles en la interfaz | P-20 | 3–4 d | | F3.4 | Importación de datos (Excel/CSV; ruta Dolibarr) | P-24 | 3–4 d | | F3.5 | Reportes exportables + comprobante de venta imprimible | P-21 | 3–4 d |

****Criterio de aceptación****: un negocio carga su catálogo desde Excel, abre caja, vende, imprime el comprobante y cierra el turno con arqueo cuadrado.

****Estado: completada (2026-09-24).**** Los cinco pasos quedaron construidos y verificados:

- ****P-15 Compras y proveedores****: dominio completo en el ERP (proveedores con borrado lógico, compras con detalle, numeración atómica `C-000001`, RLS en las tres tablas nuevas), la compra suma inventario, deja el kardex (`PURCHASE`), actualiza el costo del producto ****sin IVA**** y emite `purchase.received` por la bandeja de salida; la anulación revierte el stock (`PURCHASE\_VOID`).

- ****P-16 Sesiones de caja****: apertura con base, una sola caja abierta por negocio

(índice único parcial), ventas ligadas al turno, cierre con arqueo (esperado = base + efectivo de ventas completas – anuladas) y diferencia calculada.

- **P-20 Usuarios y roles**: correo único global en IAM (índice funcional sobre `lower(email)`), creación y edición de usuarios por el propietario (`requireAdmin`), cambio de rol, habilitar/deshabilitar con efecto inmediato en el ingreso (`USER_DISABLED`).

- **P-24 Importación de catálogo (CSV)**: detección de separador ``,``; alias de cabeceras en español e inglés, alta o actualización por SKU, stock por kardex (`IMPORT`), errores por línea con número, límite de 1000 filas; en la PWA con modal de archivo o texto pegado y resumen de creados/actualizados/errores.

- **P-21 Reportes y comprobante**: `GET /reports/sales.csv` (filtro `from/to` interpretado en la **zona horaria del negocio**, fechas ISO con offset) y `GET /reports/inventory.csv` valorizado; fechas inválidas responden 400. La PWA descarga ambos reportes y el POS imprime el comprobante de la última venta sin depender de internet.

- **Defecto corregido durante el cierre**: el ERP no traía base de datos de zonas horarias (`tzdata`), así que `DateTime.now("America/Bogota")` fallaba y el día comercial caía silenciosamente a UTC entre las 19:00 y las 23:59 locales (el mismo defecto A-01, latente en otra capa). Se añadió `tzdata`, se registró el respaldo y el humo verifica que el filtro de fechas respeta la zona del negocio.

- **Evidencia**: humo **102/102** (12 bloques, repetible en el mismo minuto), contratos **12/12**, E2E **4/4** con auditoría axe, `make ci` en verde.

Fase 4 – Diferenciadores (5–6 semanas)

| Paso | Pendiente | ID | Esfuerzo | | -- | -- | -- | -- | | F4.1 | Vertical Packs (retail, servicios, restaurantes, agro) | P-17 | 4–6 d | | F4.2 | Facturación electrónica DIAN (UBL 2.1, CUFE, QR) como puerto enchufable | P-18 | 4–6 d | | F4.3 | Notificaciones (WhatsApp/correo) + servicio de Documentos | P-19

· P-25 | 4–6 d || F4.4 | Multi-bodega y transferencias | P-22 | 2–3 d || F4.5 | Segundo factor (TOTP) del propietario | P-30 | 1–2 d || F4.6 | ADR-0012: zona horaria por negocio en la tabla `tenants` | – | 0.5 d |

****Criterio de aceptación****: dos verticales activables sin tocar el núcleo · factura

DIAN emitida en ambiente de habilitación · notificación real entregada ·

transferencia entre dos bodegas con kardex en ambas.

****Estado: en curso (2026-09-24).****

- ****F4.6 ADR-0012 · zona horaria por negocio: completado.**** `tenants.timezone`

en IAM (migración `V5`), claim `tenant\_timezone` en el token, cabecera

`x-tenant-timezone` propagada por el gateway (reescrita siempre con el valor

verificado, nunca la del cliente) y aplicada por el ERP con validación contra

la base de zonas (400 `INVALID\_TIMEZONE` si es desconocida) y respaldo

configurado. Evidencia: IAM 35 pruebas (claim), gateway 8/8 (cabecera),

ERP 5/5 de integración (precedencia), humo ****109/109**** (zona del negocio,

zona distinta honrada y zona inválida rechazada).

- ****F4.1 Vertical Packs (P-17): en curso.**** El vertical es un dato del negocio

(ADR-0013), no una bifurcación del producto:

- ****Catálogo**** en el ERP (`KuboErp.Packs`): cuatro verticales —retail,

servicios, restaurantes y agro— con terminología, IVA por defecto, si

lleva inventario y flujo de punto de venta; `GET /packs` y

`GET /packs/current`; un vertical desconocido cae al paquete por defecto

con aviso (solo afecta etiquetas, no datos).

- ****Negocio****: `tenants.vertical` en IAM (migración `V6`), claim

`tenant\_vertical`, cabecera `x-tenant-vertical` propagada por el gateway y

`GET/PATCH /tenants/me` (solo propietario o administrador) para activar un

paquete; el cambio aplica al refrescar la sesión, que la propia interfaz

dispara al guardar.

- **PWA**: contexto ``usePack`` que adapta la terminología (una tienda ve «Productos» y un restaurante «Platillos») y pantalla **Configuración** con el selector de vertical y la zona horaria.
- **Evidencia**: IAM **36 pruebas** (claim y validación ``INVALID_VERTICAL``), gateway **9/9** (cabecera y rutas), ERP puras **16/16** y 5/5 de integración, humo **114/114** (catálogo, demo en retail, vertical inválido rechazado y cambio reflejado en el token nuevo), E2E **4/4** con la pantalla nueva auditada.
- **Incremento aplicado**: el paquete ya cambia el comportamiento, no solo las etiquetas. ``products.tracks_stock`` (migración ``20260923000010``): un servicio se vende sin existencias, no mueve kardex al vender y no lo devuelve al anular; el POS no bloquea la venta ni avisa de stock, y el formulario de productos lo propone según el paquete. ``sales.table_number``: el flujo ``pos_flow: "table"`` muestra el campo de mesa en el POS, lo guarda en la venta y lo imprime en el comprobante. Evidencia: ERP integración **6/6** (servicio sin kardex y venta con mesa), humo **117/117**, E2E 4/4.
- **Siembra del catálogo de arranque**: ``POST /packs/apply`` crea los productos de arranque del paquete activo (tres por vertical, con la terminología, el IVA y la bandera de inventario del paquete) y es **idempotente**: los SKU existentes se omiten, así que aplicarlo dos veces no duplica nada. La pantalla de Configuración lo ofrece con un botón y el resultado se informa al usuario. Evidencia: ERP integración **7/7**, humo **120/120** (siembra, no duplica y omite existentes).
- **Criterio de aceptación de F4.1 cubierto**: dos verticales activables sin tocar el núcleo (probado con ``retail`` y ``restaurantes`` en el humo), con terminología, valores por defecto y comportamiento distintos.

- **F4.2 Facturación electrónica DIAN (P-18): puerto implementado.** El núcleo no conoce al proveedor: `KuboErp.Billing` es el contrato y el adaptador `Sandbox` genera UBL 2.1 con el CUF del algoritmo DIAN (SHA-384, 96 hexadecimales), el QR del catálogo y el escapado XML. La factura se persiste inmutable (`invoices`, con RLS), emitir es idempotente, una venta anulada no se factura (409 `SALE\_VOIDED`) y el nombre del emisor llega por `x-tenant-name`. Evidencia: ERP puras **20/20** y integración **9/9**, humo **125/125**, contratos **16/16** (esquema `Invoiceltem` contra la API viva). Pendiente declarado: notas crédito, firma XAdES y registro del NIT y la clave técnica ante la DIAN (trámite externo).

- **F4.5 Segundo factor TOTP del propietario (P-30): completado.** El secreto se cifra en reposo (AES-256-GCM con `KUBO\_TOTP\_ENCRYPTION\_KEY`; sin llave el servicio no arranca), la implementación es propia y verificada contra los vectores del RFC 6238, y el acceso se parte en dos: `login` responde un desafío firmado con `typ=totp` (el gateway solo acepta `typ=access`) y `POST /auth/totp/verify` lo cambia por la sesión. `setup` deja el secreto pendiente, `enable` lo confirma con el primer código y `disable` exige un código vigente; todo queda auditado. La PWA pide el código en el ingreso y la pantalla de Configuración permite configurarlo. Evidencia: IAM **46 pruebas** (vectores RFC, cifrado, desafío y ciclo completo), gateway **9/9**, humo **130/130** con un código real calculado en Python (implementación independiente), E2E 4/4, `make ci` 10/10.

- **F4.3 Notificaciones (P-19): corte implementado.** El aviso es un **puerto** (`KuboErp.Notifications`, ADR-0017): el núcleo dice qué pasó y el adaptador decide cómo se entrega; el de por defecto escribe en el **buzón del negocio** (`notifications`, con RLS), que sirve de demostración, auditoría y base para un

proveedor real. La notificación viaja en la transacción que la provoca y el aviso de stock bajo usa histéresis (avisa al ****cruzar**** el mínimo, no en cada venta). Evidencia: humo ****139/139**** con el aviso `LOW\_STOCK` y las unidades restantes. Pendiente del paso: adaptador real de WhatsApp/SMTP con entrega en segundo plano, buzón en la PWA, notificaciones de compra/resumen y el servicio de Documentos (P-25).

- ****F4.4 Multi-bodega y transferencias (P-22): modelo implementado.**** El stock por bodega es real (`warehouses`, `stock\_levels` como fuente de verdad, `products.stock` como total denormalizado mantenido en la misma transacción), el kardex gana `warehouse\_id` con índice de idempotencia por bodega y la transferencia es un ****par de movimientos atómicos**** con las reglas del ADR-0016 (origen $\neq$ destino, cantidades positivas, producto con inventario, existencia suficiente). La migración hace el backfill de bodega, niveles y kardex histórico ****antes**** de activar RLS, y crea la bodega por defecto de cada negocio (las nuevas se crean de forma perezosa). Evidencia: ERP integración ****10/10**** con PostgreSQL real, ****humo 137/137**** (transferencia, total invariante, kardex en ambas bodegas, 409 por existencia y 409 al borrar la bodega por defecto) y ****contratos 18/18**** (`WarehouseList`, `TransferList`). El backfill de la migración resultó invisible para RLS (las tablas ya tenían `FORCE`): se suspende `FORCE` durante el backfill y se restaura al terminar, con backfill idempotente. ****Completado con la PWA de bodegas/transferencias y la elección de bodega en el POS****: la venta despacha desde la bodega elegida (el ERP la valida y descuenta su nivel; una bodega ajena responde 404), con humo ****146/146**** e integración ****12/12****.

Fase 5 — Escala (backlog declarado)

| ID | Línea de trabajo | | --- | --- | | P-28 | mTLS entre gateway y servicios | | P-29 | Rotación de claves de cifrado de campo (KEK/DEK) | | - | Instalación remota (Terraform/Ansible) | | - | Multi-tenant SaaS (onboarding y zona horaria por negocio) | | - | App móvil nativa y operador de respaldos |

4. Camino crítico y calendario

flowchart LR

F0["Fase 0
2-3 h"] --> F1["Fase 1 · Confiabilidad
3 semanas"]

F1 --> F2["Fase 2 · Calidad
4 semanas"]

F2 --> F3["Fase 3 · Núcleo comercial
5-7 semanas"]

F3 --> F4["Fase 4 · Diferenciadores
5-6 semanas"]

F4 --> F5["Fase 5 · Escala
backlog declarado"]

| Hito | Duración | Semanas acumuladas | Estado | | --- | --- | --- | --- | | Fase 0 | 2-3 h | día 1 | Completada | | Fase 1 | ≈3 semanas | 1-3 | Completada | | Fase 2 | ≈4 semanas | 4-7 | Siguiendo | | Fase 3 | 5-7 semanas | 8-14 | Planificada | | Fase 4 | 5-6 semanas | 15-20 | Planificada | | **Producto comercializable** | **≈17-20 semanas** | **≈4-5 meses a 30 h/semana** | - |

Con las fases 0 y 1 cerradas, el camino restante a producto comercializable es de

≈14-17 semanas a 30 h/semana (fases 2 a 4).

5. Definición de «proyecto terminado»

1. Un negocio real opera una semana completa sin soporte presencial.
2. Cero pérdida de datos: caída del bus, del proceso o del equipo → nada se pierde (outbox + respaldos probados).
3. Aislamiento garantizado por el motor, no por el programador (RLS activo).
4. Una regresión no llega a `main`: CI con gates de seguridad y de contrato.
5. Los 5 módulos que el negocio pide (compras, caja, usuarios, comprobantes, reportes) funcionando en la PWA.
6. Documentación viva: cada decisión con su ADR, cada requisito con su prueba.

6. Riesgos y dependencias

| Riesgo | Mitigación | | --- | --- | | Push a GitLab bloqueado (falta Personal Access Token) | `GITLAB\_TOKEN=... make push`; los commits quedan locales mientras tanto | | Las pruebas del ERP no corren en el contenedor de producción (OOM con 512 MB) | Comando con `-m 3g` documentado en `07-pruebas.md`; el CI de la Fase 2 las ejecuta | | DIAN exige ser Proveedor Tecnológico

autorizado | Trámite externo; la Fase 4 avanza con el puerto y deja la habilitación como cierre | | Capturas internas de la app y video demo pendientes | Guía en `evidencia/README.md` y guion en `09-demo-guion.md`; tarea del usuario | | La Fase 3 no debe empezar sin la Fase 1 | Orden por riesgo: no se construye producto sobre una entrega de eventos no garantizada |

7. Catálogo de pendientes

El catálogo completo con IDs, riesgo y esfuerzo vive en

[`10-auditoria.md` §5](10-auditoria.md); este documento lo ordena por fase y fija

el criterio de cierre de cada una.

ADR-0001 — Microservicios con monolito modular interno

| Campo | Valor | | --- | --- | | Fecha | 2026-09-23 | | Estado | Aceptada | | Decisores | Arquitectura Kubo |

Contexto

Kubo debe cubrir cuatro dominios bien distintos (identidad, clientes, operación comercial e indicadores) y, al mismo tiempo, poder instalarse en el local de una PYME con recursos limitados (2 vCPU / 4 GB de RAM). El equipo es pequeño y el tiempo de entrega, corto.

Decisión

Se despliegan **cinco servicios independientes** (identidad, CRM, ERP, analítica, documentos) más un API Gateway, cada uno con su propia base de datos.

Dentro de cada servicio no se subdivide en más servicios: se aplica arquitectura limpia en capas (`domain`, `application`, `infrastructure`, `presentation`) y los subdominios conviven como módulos del mismo despliegue.

Alternativas consideradas

1. **Monolito único**: más simple de operar, pero incumple el requisito de microservicios y no demuestra aislamiento por dominio.

2. **Microservicios finos** (catálogo, inventario y ventas por separado): el costo de coordinación (transacciones, consistencia, red) supera el beneficio en un equipo de una persona.
3. **Microservicios con monolito modular interno** (elegida): cada servicio puede partirse más adelante sin reescribir la lógica, porque las fronteras entre módulos ya están trazadas.

Consecuencias

- **Positivas**: se cumple el requisito, el consumo de memoria es acotado (~900 MB para todo el sistema), y el dominio queda listo para escalar horizontalmente el servicio que lo necesite.
- **Negativas**: no hay transacciones distribuidas; la consistencia entre servicios es eventual y se resuelve con eventos (ver ADR-0005).
- **Regla derivada**: si dos módulos del mismo servicio se comunican con demasiada frecuencia, se refactorizan a un solo módulo; si un módulo cambia por razones distintas a su servicio, se evalúa extraerlo.

ADR-0002 — Polyrepo con contratos como fuente de verdad

| Campo | Valor | | -- | -- | | Fecha | 2026-09-23 | | Estado | Aceptada |

Contexto

El requisito del proyecto exige un repositorio separado por cada parte del software. Además, cada servicio usa un lenguaje y un ecosistema distinto, con herramientas de construcción incompatibles entre sí.

Decisión

Un repositorio por componente (nueve en total): gateway, cuatro servicios,

PWA, infraestructura, documentación y el repositorio raíz de workspace. La

comunicación entre repos se gobierna por **contratos explícitos**:

- HTTP: contrato documentado en `kubo-docs/03-api.md` con ejemplos ejecutables.

La **especificación OpenAPI 3.1** en archivo (y las pruebas de contrato

automatizadas) quedan para la Fase 2: hoy el contrato se verifica con la prueba

de humo end-to-end, que ejerce todos los endpoints desde el gateway.

- Eventos: sobre estable versionado (`event\_id`, `event\_type`, `version`,
`occurred\_at`, `tenant\_id`, `data`).

- Base de datos: cada servicio es dueño exclusivo de su esquema; **ningún**
servicio consulta la base de otro

Consecuencias

- **Positivas**: cada servicio se construye, versiona y despliega por separado;
un fallo de toolchain en un lenguaje no bloquea a los demás; los equipos
pueden crecer sin fricción de monorepo.

- **Negativas**: los cambios de contrato exigen coordinación entre repos. Se
mitiga con versionado semántico del contrato y una prueba de humo que valida
el flujo completo antes de cada entrega.

- **Regla derivada**: un cambio incompatible en un contrato obliga a subir la
versión del evento o del endpoint, nunca a modificar el significado del
existente.

ADR-0003 — Multi-tenancy híbrido: `tenant\_id` + RLS activo

| Campo | Valor | | --- | --- | | Fecha | 2026-09-23 | | Actualizado | 2026-09-24 (RLS activado en la Fase 1) |
| Estado | Aceptada |

Contexto

Kubo se vende como software autoalojable: cada negocio instala su propia copia y no paga suscripción. Sin embargo, el mercado exige más adelante una versión en la nube con varias empresas en un mismo despliegue. Cambiar de modelo después es costosísimo si el modelo de datos no lo previó.

Decisión

Modelo híbrido: el código y el esquema son multi-tenant desde el día uno (todas las tablas de negocio llevan `tenant_id`` y todas las consultas filtran por él), pero **el despliegue por defecto es de un solo negocio por instalación**.

Como segunda barrera, **RLS está activo con `FORCE``** en las tres bases PostgreSQL. Cada petición abre una transacción y fija `app.tenant_id`` mediante un interceptor propio de cada lenguaje; las operaciones que cruzan negocios por diseño (autenticación por correo, cadena de auditoría global, semilla) usan la marca `app.system``. El detalle y las consecuencias están en [ADR-0010](ADR-0010-rls-activo.md).

Activación de RLS (Fase 1)

La activación se pospuso en el MVP por una razón concreta: activar RLS sin fijar la variable de sesión en cada transacción deja las consultas sin resultados —un fallo silencioso—. La Fase 1 aportó lo que faltaba:

1. Interceptores de transacción en los tres servicios (Java, Rails y Phoenix).
2. Políticas versionadas en migraciones (`V3__enable_rls.sql`` en IAM, `20260923000003_enable_rls.rb`` en CRM y `20260923000004_enable_rls.exs`` en ERP).
3. La distinción explícita entre contexto de negocio y contexto de sistema.
4. Verificación automática en `make smoke``: sin contexto **0 filas**, con contexto > 0, en las tres bases.

Consecuencias

- **Positivas**: el mismo binario sirve para autoalojamiento y para nube; no hay que migrar el esquema para cambiar de modelo de negocio. El aislamiento ya no depende de la disciplina del código: lo impone el motor.
- **Negativas**: toda transacción nueva debe fijar su contexto; los interceptores cubren el camino normal y los caminos de sistema lo hacen explícito. Es la regla a recordar al agregar código.
- **Siguiente paso**: zona horaria por negocio en la tabla `tenants` (ADR-0012, Fase 4) para el despliegue SaaS multi-tenant.

ADR-0004 — Poliglotismo deliberado: un lenguaje por contexto

| Campo | Valor | | --- | --- | | Fecha | 2026-09-23 | | Estado | Aceptada |

Contexto

El proyecto exige al menos cuatro microservicios con un mínimo de tres tecnologías distintas. La tentación es elegir un lenguaje y repetirlo por comodidad, pero eso convierte el requisito en un ejercicio vacío.

Decisión

Cada servicio se implementa en el lenguaje cuya **forma natural** encaja con su problema, y esa elección se documenta:

| Servicio | Lenguaje | Por qué ese y no otro | | --- | --- | --- | | Identidad | Java 21 + Spring Boot 4 | El ecosistema de seguridad más maduro: Spring Security, validación, JPA, migraciones. Aquí la corrección importa más que la novedad. | | CRM | Ruby 3.4 + Rails 8 | Velocidad de evolución del modelo comercial. ActiveRecord y las migraciones permiten iterar el esquema de clientes en minutos. | | ERP | Elixir 1.17 + Phoenix 1.8 | La venta es la operación crítica: concurrencia masiva, latencia predecible y tolerancia a fallos con árboles de supervisión de OTP. | | Analítica | Python 3.13 + FastAPI | Agregaciones y, más adelante, pronóstico de demanda. El ecosistema de datos y el motor de agregación de MongoDB encajan sin esquema rígido. | | Gateway | TypeScript + NestJS | Capa de

borde con validación de JWT, límites de tasa y enrutamiento: el mismo lenguaje del frontend reduce el cambio de contexto. | | PWA | React 19 + Vite | Interfaz instalable, offline-first y con tipado compartido. |

Consecuencias

- **Positivas**: el equipo aprende cinco ecosistemas reales; cada servicio usa la herramienta idónea; el requisito se cumple con sentido y no por relleno.
- **Negativas**: cinco toolchains, cinco pipelines, cinco formas de hacer pruebas. Se mitiga con un contrato común (ADR-0002), con la misma estructura de carpetas y con la prueba de humo que valida el sistema completo.
- **Regla derivada**: ningún servicio se reescribe en otro lenguaje sin un ADR que lo justifique con métricas (latencia, costo, mantenibilidad).

ADR-0005 — Eventos con RabbitMQ y publicador tolerante a fallos

| Campo | Valor | | -- | -- | | Fecha | 2026-09-23 | | Estado | Aceptada (outbox implementado en la Fase 1; ver ADR-0009) |

Contexto

Cuando se registra una venta, varios procesos deben reaccionar: analítica proyecta el indicador, notificaciones avisa al cliente, cartera actualiza el saldo. Si el ERP llamara a cada uno por HTTP, una caída de analítica rompería la venta y el acoplamiento crecería con cada consumidor nuevo.

Decisión

1. El ERP publica **eventos de dominio** en el exchange de tipo **topic** ``kubo.events``, con sobre estable y versionado (``sale.created` v1`).
2. Los consumidores **deduplican por `event\_id`**: analítica usa un índice único en MongoDB, de modo que una reentrega del broker no cuenta la venta dos veces.

3. Los mensajes que no se pueden procesar van a una ****cola de mensajes muertos**** (DLQ), nunca se pierden ni bloquean la cola principal.
4. ****La venta nunca depende del bus****: la publicación es asíncrona y tolerante a fallos. Si RabbitMQ está caído, se registra la advertencia y el negocio sigue operando.

Lo que queda pendiente y por qué

El patrón ***transaccional outbox*** (guardar el evento en la misma transacción de la venta y publicarlo después desde una tabla de salida) elimina la ventana en la que un evento puede perderse si el proceso muere justo después del ``commit``. En el MVP se asume esa pérdida eventual y se documenta; la implementación del outbox es el primer trabajo de la Fase 1 (P-01, ADR-0009).

Consecuencias

- ****Positivas****: desacoplamiento real (analítica puede caer sin afectar la caja), reacción en segundos y camino abierto a nuevos consumidores sin tocar el ERP.
- ****Negativas****: consistencia eventual (el tablero tarda milisegundos en reflejar la venta) y, en el MVP, posibilidad de perder un evento ante una caída en el instante exacto del ``commit``.
- ****Nota técnica****: se usa ``amqp` 4.x`; la serie 3.x depende de ``rabbit_common` 3.12`, que no compila en Erlang/OTP 27.

ADR-0006 — Cifrado de datos personales con AES-256-GCM e índice ciego

| Campo | Valor | | -- | -- | | Fecha | 2026-09-23 | | Estado | Aceptada |

Contexto

El CRM guarda datos personales de clientes (documento, teléfono), protegidos por

la Ley 1581 de 2012 (habeas data). Si alguien obtiene una copia del respaldo o accede a la base, esos datos no deben quedar legibles. Al mismo tiempo, el negocio necesita **buscar por documento** con precisión.

Decisión

1. **Cifrado a nivel de campo** con **AES-256-GCM** (confidencialidad + autenticidad: cualquier manipulación del texto cifrado se detecta y el valor se rechaza). Formato almacenado: ``base64(iv || tag || ciphertext)``.
2. **Índice ciego** con HMAC-SHA256 sobre el valor normalizado, en una columna aparte. Permite buscar por igualdad sin descifrar y sin exponer el dato.
3. **Minimización**: los listados devuelven el valor enmascarado (``*****432``); solo el detalle de un cliente concreto revela el valor completo.
4. Las claves llegan por variable de entorno (``KUBO_FIELD_ENCRYPTION_KEY``, ``KUBO_BLIND_INDEX_KEY``) y **nunca** están en el repositorio.

Alternativas consideradas

- **Cifrar toda la base** (TDE o disco cifrado): no está disponible en PostgreSQL comunitario y no protege contra un volcado lógico.
- **Guardar el dato en claro y confiar en los permisos**: insuficiente ante un respaldo filtrado, que es el escenario real más frecuente.
- **Cifrado determinista** (mismo texto cifrado para el mismo valor): permitiría buscar, pero filtra información por comparación. Se prefiere el índice ciego.

Consecuencias

- **Positivas**: un volcado de la tabla no revela documentos ni teléfonos; el GCM detecta manipulaciones; la búsqueda sigue siendo exacta y rápida.
- **Negativas**: no se puede buscar por coincidencia parcial sobre el campo cifrado (solo por igualdad) y rotar la clave exige re-cifrar los registros.

- **Siguiente paso**: envoltura de claves (KEK/DEK por negocio) con un KMS para la versión en la nube.

ADR-0007 — JWT RS256 con JWKS: la llave privada no sale del servicio de identidad

| Campo | Valor | | -- | -- | | Fecha | 2026-09-23 | | Estado | Aceptada |

Contexto

Varios servicios necesitan saber quién es el usuario y a qué negocio pertenece.

La opción cómoda es que cada servicio comparta un secreto simétrico y verifique los tokens por su cuenta, pero eso reparte el secreto por todo el sistema: quien comprometa un servicio puede **emitir** tokens válidos para cualquier usuario.

Decisión

1. El servicio de identidad genera un par de llaves **RSA** y firma los access tokens con **RS256**. La llave privada vive únicamente ahí (o llega por `KUBO_JWT_PRIVATE_KEY` en producción).
2. Publica la llave pública en un documento **JWKS**.
3. El **API Gateway** es el único que valida la firma, contra el JWKS, con caché y refresco automático ante rotación de llave.
4. Tras validar, el gateway **elimina cualquier cabecera** `X-User-` enviada por el cliente e inyecta la identidad real antes de reenviar la petición.
5. Los servicios internos confían en esas cabeceras porque **no son alcanzables desde fuera**: solo existen en la red privada de contenedores.

Consecuencias

- **Positivas**: comprometer un servicio de negocio no permite emitir tokens; el gateway centraliza la política de acceso; los servicios no cargan librerías de

criptografía ni conocen el formato del token.

- **Negativas**: el gateway es un punto único de fallo y un salto adicional de red. Se mitiga con healthchecks, límites de memoria y la posibilidad de escalarlo horizontalmente sin estado (los límites de tasa viven en Redis).
- **Siguiente paso**: mTLS entre gateway y servicios para la versión en clúster.

ADR-0008 — Alcance declarado del MVP y recortes conscientes

| Campo | Valor | | --- | --- | | Fecha | 2026-09-23 | | Estado | Aceptada |

Contexto

La entrega del MVP ocurre en una jornada. Es imposible cubrir un ERP/CRM de producción completo, y fingir lo contrario sería peor que recortar con criterio.

Decisión

Se prioriza lo que **demuestra arquitectura funcionando de extremo a extremo** y se documenta lo que queda fuera, con su ruta de implementación.

Incluido y funcionando

- Cuatro microservicios + gateway + PWA, con base de datos por servicio.
- JWT RS256 con JWKS, rotación de refresh tokens con detección de reuso, contraseñas con BCrypt, auditoría encadenada.
- Cifrado AES-256-GCM de datos personales con índice ciego y enmascarado.
- Venta transaccional con bloqueo pesimista de inventario y kardex.
- Eventos de dominio por RabbitMQ con consumidor idempotente y DLQ.
- Tablero con agregaciones sobre MongoDB y PWA instalable con cola offline.
- Aislamiento por negocio verificado por prueba automática.

Fuera del MVP (con ruta definida)

| Recorte | Por qué | Cuándo | | --- | --- | --- | | *Transactional outbox* | Requiere tabla de salida y publicador de barrido; el MVP ya no bloquea la venta por el bus | Fase 1 (P-01) | | Activación de RLS | Necesita interceptor de transacción probado con concurrencia | Fase 1 (P-02) | | Refresh token en cookie httpOnly | Exige BFF con cookies; hoy vive en el navegador (documentado) | Fase 1 (P-03) | | Recuperación de contraseña por correo | Requiere proveedor de correo y plantillas | Fase 1 (P-04) | | Facturación electrónica DIAN (UBL 2.1, CUFE, QR) | Requiere ser Proveedor Tecnológico autorizado; el puerto de facturación queda preparado | Fase 4 (P-18) | | Multi-rubro (Vertical Packs) | El MVP demuestra un vertical (retail) completo | Fase 4 (P-17) | | Kubernetes y Terraform | El despliegue objetivo es un local con Docker Compose | Fase 5 |

Consecuencias

- **Positivas**: el sistema se entrega funcionando y verificado, con un mapa honesto de lo que falta; nada se presenta como terminado sin estarlo.
- **Negativas**: el producto no está listo para un negocio real sin antes completar la Fase 1 (outbox, RLS, cookie httpOnly, recuperación de contraseña).
- **Regla derivada**: ningún recorte se oculta en la documentación; cada uno aparece en este ADR y en la matriz de trazabilidad.

ADR-0009 — Outbox transaccional para la entrega de eventos

| Campo | Valor | | --- | --- | | Fecha | 2026-09-24 | | Estado | Aceptada (Fase 1) | | Relacionada | ADR-0005 (eventos con RabbitMQ) |

Contexto

La venta se confirmaba en PostgreSQL y **después** se publicaba `sale.created` en RabbitMQ. Entre el `commit` y la publicación existe una ventana en la que el proceso puede morir: la venta ya se cobró al cliente, pero el evento se pierde y el tablero jamás se entera. Es el único punto del sistema donde se podía perder un dato que el usuario ya confirmó.

Publicar antes del `commit` tampoco sirve: un rollback dejaría un evento de una venta que no ocurrió.

Decisión

Patrón ****transactional outbox****:

1. La venta y su evento se guardan en la ****misma transacción****: ``Sales.insert_sale`` inserta el sobre completo en ``outbox_events`` (``PENDING``) antes del ``commit``.

Si la transacción se revierte, el evento desaparece con ella.

2. Un ****publicador de barrido**** (``KuboErp.Events.Publisher``) toma lotes con ``FOR UPDATE SKIP LOCKED`` —seguro con varias instancias del ERP— y los entrega a RabbitMQ. Una venta lo despierta (``kick/0``); además barre cada 500 ms.

3. Entrega ****al-menos-una-vez****: si el proceso muere después de publicar y antes de marcar la fila, el evento se reentrega. Los consumidores ya deduplican por ``event_id`` (índice único en MongoDB).

4. Reintentos con espera exponencial (2, 4, 8... hasta 300 s). Tras 10 intentos el evento queda ``FAILED``, visible en la sonda de salud: un evento que no se puede entregar es un incidente, no un detalle.

5. La conexión AMQP se ****monitorea**** (``Process.monitor``): si el bus cae, el publicador vuelve a estado desconectado y los eventos esperan en la bandeja.

Alternativas descartadas

| Alternativa | Por qué no | | -- | -- | | Reintentos en memoria tras el fallo de publicación | No sobreviven a la muerte del proceso: la ventana seguía abierta | | Transacción distribuida (2PC/XA) con el broker | Complejidad alta y soporte limitado en los cuatro lenguajes | | CDC (Debezium) sobre el WAL | Una pieza más de infraestructura para el mismo resultado |

Consecuencias

- ****Positivas****: cero pérdida de eventos confirmados; la caja nunca depende del bus; el tablero se recupera solo tras una caída; trazabilidad de reintentos.

- ****Negativas****: tabla operativa que crece (requiere política de purga en Fase 2); entrega duplicada posible, asumida por consumidores idempotentes.

- `outbox\_events` queda fuera de RLS: es una tabla operativa que el publicador lee cruzando negocios para entregar los eventos.

Verificación

- `make bus-drill` (4/4): con RabbitMQ detenido la venta se registra, el evento queda `PENDING`, y al volver el bus se publica y llega a analítica.
- `make smoke`: el evento de la venta queda en la bandeja y se publica.
- La sonda de salud del ERP expone `outbox: {pending, published, failed}`.

ADR-0010 — RLS activo con interceptor de transacción

| Campo | Valor | | --- | --- | | Fecha | 2026-09-24 | | Estado | Aceptada (Fase 1) | | Relacionada | ADR-0003 (multi-tenancy híbrido) |

Contexto

El aislamiento entre negocios dependía de que **toda** consulta recordara filtrar por `tenant\_id`. Un olvido no se detecta: no falla, simplemente muestra datos de otro negocio. La política de *row level security* existía escrita desde el MVP, pero desactivada porque activarla sin un interceptor rompe autenticación y semilla.

Decisión

RLS activo en las tres bases PostgreSQL, con **FORCE** (el rol de la aplicación es dueño de las tablas; sin FORCE PostgreSQL lo eximiría de las políticas).

1. Cada transacción fija su contexto con

`select set\_config('app.tenant\_id', '<uuid>', true)` — local a la transacción, imposible de filtrar a otra petición que reutilice la conexión.

2. **Interceptores por lenguaje**:

- IAM (Java): `TenantRlsFilter` envuelve cada petición en una transacción.

- CRM (Rails): ``around_action`` en ``ApplicationController``.

- ERP (Phoenix): ``action/2`` en ``KuboErpWeb.controller``.

3. **Marca ``app.system``** para operaciones que cruzan negocios por diseño:

autenticación por correo (el negocio aún no se conoce), rotación de tokens,

cadena de auditoría global, semilla y respuesta a incidentes de seguridad.

Sin ella, login y recuperación de contraseña serían imposibles.

4. Alcance por tabla:

- ``sale_items`` no tiene ``tenant_id``: su política hereda el negocio por la venta.

- ``outbox_events`` queda fuera: tabla operativa que el publicador lee cruzando negocios (ver ADR-0009).

- ``tenants`` queda sin RLS: el registro debe comprobar la unicidad global del slug.

5. Las transacciones ``REQUIRES_NEW`` **no heredan** el contexto de la exterior:

``SecurityIncidentService`` y ``AuditService`` fijan su propia marca. Sin esto, la

revocación de la familia de tokens actualizaba cero filas **en silencio** —el

sistema detectaba el robo y no cerraba nada—.

Consecuencias

- El aislamiento pasa a ser garantía del motor, no del programador: una consulta sin contexto devuelve **cero filas**.

- Toda transacción nueva debe fijar contexto; es la regla a recordar al agregar código. Los interceptores cubren el camino normal; lo explícito queda en los caminos de sistema.

- El modo sistema es una capacidad deliberada, acotada a identidad y auditoría.

- El rendimiento no se degrada de forma perceptible (las políticas son comparaciones por índice sobre ``tenant_id``).

Verificación

- ``make smoke``: consulta sin contexto = 0 filas y con contexto > 0 en IAM, CRM y ERP.
- El humo comprueba además que el reuso de un token revoca la **familia completa**.
- ``04-seguridad.md`` §4 documenta las políticas vigentes.

ADR-0011 — BFF como capa formal del gateway

| Campo | Valor | | --- | --- | | Fecha | 2026-09-24 | | Estado | Aceptada (Fase 2) | | Relacionada | ADR-0007 (JWT RS256 con JWKS) |

Contexto

El patrón *Backend For Frontend* apareció dos veces en el gateway sin haberse declarado como decisión: primero con ``GET /api/v1/dashboard/overview`` (la vista compuesta del tablero) y después con el BFF de autenticación, que guarda el refresh token en una cookie ``httpOnly``. Sin una decisión explícita, cada nueva necesidad de la PWA corría el riesgo de resolverse con lógica de presentación repartida entre el navegador y los microservicios.

Decisión

El gateway es la **capa BFF formal** del sistema. Toda composición o adaptación pensada para la PWA vive ahí, no en el navegador ni en los servicios de negocio:

1. **Composición de lecturas** (``/dashboard/overview``): el gateway consulta los servicios en paralelo (``Promise.allSettled``) y entrega una sola respuesta. Si una vista falla, el tablero llega parcial y lo informa en ``unavailable``.
2. **Adaptación de credenciales** (``login`/`refresh`/`logout``): el refresh token se transforma en cookie ``httpOnly`` + ``SameSite=Strict`` y desaparece del cuerpo. El navegador nunca lo ve.
3. **Reglas del BFF**:
 - Los servicios de negocio conservan contratos limpios y sin conocimiento de la interfaz.

- El BFF no duplica reglas de dominio: solo orquesta y adapta forma.
- Las rutas compuestas se declaran en ``GATEWAY_OWNED_PATHS`` y no se reenvían.

Consecuencias

- ****Positivas****: un solo lugar para la composición y la seguridad de sesión; los microservicios pueden evolucionar sin romper a la PWA; la cookie ``httpOnly`` es posible sin tocar los servicios de negocio.
- ****Negativas****: el gateway gana responsabilidad y debe probarse como capa propia (contratos OpenAPI + E2E + humo); una caída del gateway afecta a todas las vistas compuestas (mitigado con entrega parcial).

Verificación

- ``make smoke``: la vista compuesta trae las 7 vistas en una petición y el login no expone el refresh token.
- ``make contracts``: el contrato OpenAPI declara ``additionalProperties: false`` en ``LoginResponse``; un ``refreshToken`` filtrado rompería la validación.
- ``make e2e``: el navegador inicia sesión y navega sin leer nunca el token de refresco.